From Continuous to Discrete: Building an Individual-Based Model

Benjamin Chesworth

Supervisor: Andrew Rowntree
11303777
MATH30000

May 2026

Contents

Chapter 1

Introduction

1.1 Abstract

The Lotka-Volterra equations are celebrated because they present a simplistic model of how predator and prey interact. In this report we will build on this by discretising the equations, and by the end, building an individual-based model. We will analyse how this model differs from Lotka-Volterra, with the aim to answer: is its simplicity justified?

1.2 Road Map

We start by introducing the Lotka-Volterra equations, if somewhat unconventionally. We split the commonly used variables into ones with tangible meaning for an individual-based model. These variables will become useful when we come to compare different models as we go along. We analyse these equations to understand their characteristics. We then take steps to convert the equations into an individual-based model simulation, one which treats each animal as its own separate entity, as opposed to amalgamating them as Lotka-Volterra does.

This path starts by discretising the equations, then converts them into a stochastic model. From there we design an individual-based model that allows animals to move on a grid, and interact with each other when they meet.

This unpacking and modification of Lotka-Volterra helps us understand what the equations really mean. In this report, let us view this model through a different lens [5].

Chapter 2

Solving the Lotka-Volterra Equations

2.1 Deriving the equations

The LV equations attempt to model the change in population of two species over time. These two species have a predator-prey relationship to each other. We assume that the prey species are the predators' only food source and this species of predator is the only species that is a predator to the prey. Therefore, there are no other animals that affect our model.

Let us call the number of prey x and the number of predators y. In our model we assume that these numbers are continuous for simplicity. The larger the size of the population we are modelling, the more valid this assumption is as we can let one prey in our model represent 1000 prey in the real world, say.

In our model x and y are functions of time so at time t, $x = x(t)$ and $y = y(t)$. Over time, we assume that the prey population reproduces at a rate proportional to its population size. This is based on the assumption that each individual prey has the same rate of reproduction. We model this rate of reproduction continuously.

Further, we assume that each prey's rate of reproduction does not change over time. This relies on the assumption that the reproduction rate of an individual prey does not change with respect to changing prey population size. This is a drawback of the model because, without predator interference, the prey's population is allowed to grow unbounded and exponentially, which is unrealistic.

Let $f(x)$ be the rate of increase of the prey population from reproduction. We have assumed that it is proportional to the prey population size x, and so we define α as the proportionality constant. Hence

$$f(x) = \alpha x. \tag{2.1}$$

As $\alpha = f(x)/x$, this parameter represents the rate of reproduction per prey. We will assume that $\alpha > 0$.

Next, we assume that the death rate of predators only depends on its population size y, and define it as $g(y)$. We assume the rate is proportional to its population size, and define the proportionality constant to be γ. Hence

$$g(y) = \gamma y \tag{2.2}$$

We next want to add to our model the effect of the predators and prey on each other's population sizes. We model the rate at which prey get eaten as a function of x and y, $h(x, y)$. We assume that a predator eating a prey is synonymous with the prey dying, meaning that a predator cannot feed on a prey without killing it.

We assume that there is a probability β_1 that a predator will kill a prey when it meets it (and that this probability is constant). We multiply this by the rate at which prey and predators meet, $h_1(x, y)$, to get the rate of prey death,

$$h(x, y) = \beta_1 h_1(x, y). \tag{2.3}$$

We assume that the predators and prey are well-mixed in their environment meaning that each pair of an individual predator and prey have the same likelihood of meeting. We define the meeting rate of a given pair as ρ. As there are $x \cdot y$ individual distinct predator-prey pairs, we model the rate at which predators and prey meet as

$$h_1(x, y) = \rho x y \tag{2.4}$$

$$\implies h(x, y) = \beta_1 \rho x y. \tag{2.5}$$

We then define a new constant

$$\beta = \beta_1 \rho \tag{2.6}$$

so that

$$h(x, y) = \beta x y. \tag{2.7}$$

Next we want to derive the predator's birth rate, $i(x, y)$. We use the idea that a predator would need to consume a certain number of prey in order to have the biomass necessary to reproduce (this number could be under one if the prey is much larger than the predators). We define a parameter σ as the average number of prey eaten per predator birth. (We say 'the average number' so that we can use it in our continuous model; we will discuss this in more detail later.) For example, $\sigma = 5$ would mean a predator has to eat, on average, five prey to reproduce. As we model the populations as continuous, we can say that $1/\sigma$ of a predator is born every time a prey is eaten. We then have that the rate of predator reproduction is

$$i(x, y) = \frac{1}{\sigma} h(x, y) = \frac{1}{\sigma} \beta_1 \rho x y. \tag{2.8}$$

We define a new parameter

$$\delta = \frac{1}{\sigma}\beta = \frac{1}{\sigma}\beta_1\rho \tag{2.9}$$

so that

$$i(x, y) = \delta xy. \tag{2.10}$$

We assume that only the mechanisms outlined above cause an effect on the population sizes of the animals. So, over time, a prey's population increases by its birth rate $f(x)$ and decreases by the number eaten by predators $h(x, y)$. The predators' population decreases by its death rate $g(y)$ and increases by its birth rate $i(x, y)$. We can then find how the population sizes change by taking small time steps, Δt,

$$x(t + \Delta t) = x(t) + \Delta t(f(x) - h(x, y)) \text{ and} \tag{2.11}$$
$$y(t + \Delta t) = y(t) + \Delta t(-g(y) + i(x, y)) \tag{2.12}$$

which implies that

$$\frac{x(t + \Delta t) - x(t)}{\Delta t} = f(x) - h(x, y) \text{ and} \tag{2.13}$$
$$\frac{y(t + \Delta t) - y(t)}{\Delta t} = -g(y) + i(x, y). \tag{2.14}$$

Then by using the definition of the derivative of a function (equation A.1), and making the change in time infinitesimally small, $\Delta t \to 0$, we get two differential equations describing the change in the populations x and y over time,

$$\dot{x} = f(x) - h(x, y) \text{ and} \tag{2.15}$$
$$\dot{y} = -g(y) + i(x, y), \tag{2.16}$$

using dot notation to signify derivatives with respect to time. Then, by subbing in equations 2.1, 2.2, 2.7, and 2.10, we get

$$\dot{x} = \alpha x - \beta xy \text{ and} \tag{2.17}$$
$$\dot{y} = -\gamma y + \delta xy. \tag{2.18}$$

These are the famous Lotka-Volterra equations [1]. We can solve these numerically with initial conditions $x(0) = x_0$ and $y(0) = y_0$.

2.2 Analysing the Equations

2.2.1 Stationary Points

We firstly want to find the stationary points of the system. We set $\dot{x} = \dot{y} = 0$ which gives us

$$-\gamma y + \delta xy = 0 \text{ and } \alpha x - \beta xy = 0 \tag{2.19}$$

$$\Rightarrow y(-\gamma + \delta x) = 0 \text{ and } x(\alpha - \beta y) = 0. \tag{2.20}$$

Thus, the stationary points are

$$(0,0) \text{ or } (\frac{\gamma}{\delta}, \frac{\alpha}{\beta}). \tag{2.21}$$

We next want to find out the stability of these stationary points. We can represent the ODEs as $\dot{x} = f(x,y)$ and $\dot{y} = g(x,y)$. As these do not depend on time, they are a coupled system of autonomous equations.

To analyse the stability of stationary points for autonomous equations in general, we look at two small perturbations c and d from a fixed point (x_*, y_*) and how x and y change over time after this small perturbation.

We define the perturbations in the x and y directions from the fixed point over time caused by the initial perturbations as ϵ and δ respectively such that

$$\epsilon(t) = x(t) - x_* \text{ and } \delta(t) = y(t) - y_* \tag{2.22}$$

where $\epsilon(0) = c$ and $\delta(0) = d$.

We have that

$$\dot{\epsilon} = \frac{d}{dt}(x(t) - x_*) = \dot{x} \text{ and} \tag{2.23}$$

$$\dot{\delta} = \frac{d}{dt}(y(t) - y_*) = \dot{y} \tag{2.24}$$

since x_* and y_* are constants. Therefore $\dot{\epsilon} = f(x,y)$ and $\dot{\delta} = g(x,y)$, and since $x = x_* + \epsilon$ and $y = y_* + \delta$, we know that

$$\dot{\epsilon} = f(x_* + \epsilon, y_* + \delta) \text{ and} \tag{2.25}$$

$$\dot{\delta} = g(x_* + \epsilon, y_* + \delta). \tag{2.26}$$

In our stability analysis we assume t to be small meaning that we can take the Taylor Expansion around $t = 0$ for f and g. So we have

$$\dot{\epsilon} = f(x_* + \epsilon, y_* + \delta) = f(x_* + \epsilon(0), y_* + \delta(0)) \tag{2.27}$$

$$+ f_x(x_* + \epsilon(0), y_* + \delta(0)) \cdot (x - x(0)) \tag{2.28}$$

$$+ f_y(x_* + \epsilon(0), y_* + \delta(0)) \cdot (y - y(0)) \tag{2.29}$$

$$+ ... \tag{2.30}$$

We have taken $\epsilon(0)$ and $\delta(0)$ to be small enough such that $x_* + \epsilon(0) = x_*$ and $y_* + \epsilon(0) = y_*$. So $f(x_* + \epsilon(0), y_* + \delta(0)) = f(x_*, y_*) = 0$.

We can also neglect higher order terms of ϵ and δ as they will be negligible in size. We thus have

$$f(x_* + \epsilon, y_* + \delta) = f_x(x_*, y_*) \cdot \epsilon + f_y(x_*, y_*) \cdot \delta. \tag{2.31}$$

We can similarly Taylor expand $\dot{\delta} = g(x_* + \epsilon, y_* + \delta)$ so that we are left with

$$\dot{\epsilon} = f_x(x_*, y_*) \cdot \epsilon + f_y(x_*, y_*) \cdot \delta \text{ and} \tag{2.32}$$

$$\dot{\delta} = g_x(x_*, y_*) \cdot \epsilon + g_y(x_*, y_*) \cdot \delta. \tag{2.33}$$

Using the notation f_x^* to represent $f_x(x_*, y_*)$, we can write the above equations as

$$\frac{d}{dt}\begin{pmatrix}\epsilon\\ \delta\end{pmatrix} = \begin{pmatrix}f_x^* & f_y^*\\ g_x^* & g_y^*\end{pmatrix}\begin{pmatrix}\epsilon\\ \delta\end{pmatrix} \tag{2.34}$$

which is a system of first order linear ODEs.

To solve these linear ODEs, we use the ansatz $\begin{pmatrix}\epsilon\\ \delta\end{pmatrix} = \boldsymbol{v}e^{\lambda t}$ for some $\boldsymbol{v} \in \mathbb{R}^2 \backslash \{\mathbf{0}\}$ and $\lambda \in \mathbb{C}$. Then we let $J^* = \begin{pmatrix}f_x^* & f_y^*\\ g_x^* & g_y^*\end{pmatrix}$. So we have

$$\frac{d}{dt}(\boldsymbol{v}e^{\lambda t}) = J^* \cdot \boldsymbol{v}e^{\lambda t} \tag{2.35}$$

$$\implies \lambda e^{\lambda t}\boldsymbol{v} = J^*\boldsymbol{v}e^{\lambda t} \tag{2.36}$$

$$\implies \lambda \boldsymbol{v} = J^*\boldsymbol{v}. \tag{2.37}$$

This is an eigenvalue problem. We use the characteristic equation $\det(J^* - \lambda I) = 0$ which gives us

$$\begin{vmatrix} f_x^* - \lambda & f_y^* \\ g_x^* & g_y^* - \lambda \end{vmatrix} = 0. \tag{2.38}$$

So

$$(f_x^* - \lambda)(g_y^* - \lambda) - g_x^* f_y^* = 0 \tag{2.39}$$

$$\implies \lambda^2 + (-(f_x^* + g_y^*))\lambda + (f_x^* g_y^* - g_x^* f_y^*) = 0. \tag{2.40}$$

Hence, using the quadratic formula,

$$\lambda = \frac{(f_x^* + g_y^*) \pm \sqrt{(f_x^* + g_y^*)^2 - 4(f_x^* g_y^* - g_x^* f_y^*)}}{2} \tag{2.41}$$

$$= \frac{f_x^* + g_y^* \pm \sqrt{f_x^{*2} + g_y^{*2} + 4g_x^* f_y^* - 2f_x^* g_y^*}}{2} \tag{2.42}$$

In the case of our equations

$$\dot{x} = f(x, y) = \alpha x - \beta xy \tag{2.43}$$
$$\dot{y} = g(x, y) = -\gamma y + \delta xy \tag{2.44}$$

we have that

$$f_x = \alpha - \beta y \qquad f_y = -\beta x \tag{2.45}$$
$$g_x = \delta y \qquad g_y = -\gamma + \delta x. \tag{2.46}$$

At $(x_*, y_*) = (0, 0)$,

$$f_x^* = \alpha \qquad f_y^* = 0 \tag{2.47}$$
$$g_x^* = 0 \qquad g_y^* = -\gamma. \tag{2.48}$$

Subbing these into 2.42 we get

$$\lambda = \frac{\alpha - \gamma \pm \sqrt{\alpha^2 + \gamma^2 + 0 - 2\alpha(-\gamma)}}{2} \tag{2.49}$$
$$= \frac{\alpha - \gamma \pm (\alpha + \gamma)}{2} \tag{2.50}$$

and so $\lambda = \alpha$ or $\lambda = -\gamma$.

The general solution for a system of linera differential equations is a linear combination of its eigenvectors and eigenvalues.

So for $\boldsymbol{v_1}, \boldsymbol{v_2} \in \mathbb{R}^2 \setminus \{\mathbf{0}\}$ being the eigenvectors we have that

$$\begin{pmatrix} \epsilon \\ \delta \end{pmatrix} = c_1 \boldsymbol{v_1} e^{\alpha t} + c_2 \boldsymbol{v_2} e^{-\gamma t} \tag{2.51}$$

where $c_1, c_2 \in \mathbb{R}$.

Assume an initial perturbation where $c_1 \neq 0$, ie

$$\begin{pmatrix} \epsilon(0) \\ \delta(0) \end{pmatrix} = c_1 \boldsymbol{v_1} + c_2 \boldsymbol{v_2}, \text{where } c_1 \neq 0. \tag{2.52}$$

Then $|c_1 \boldsymbol{v_1}| > 0$ and so (as $\alpha > 0$), $|c_1 \boldsymbol{v_1} e^{\alpha t}| \to \infty$ as $t \to \infty$ and $|c_2 \boldsymbol{v_2} e^{-\gamma t}| \to 0$.

Hence the perturbation vector $\left|\begin{matrix} \epsilon \\ \delta \end{matrix}\right| \to \infty$ as $t \to \infty$, and so the fixed point $(0, 0)$ is unstable.

At $(x_*, y_*) = (\frac{\gamma}{\delta}, \frac{\alpha}{\beta})$,

$$f_x^* = \alpha - \beta(\frac{\alpha}{\beta}) = \alpha - \alpha = 0 \tag{2.53}$$

$$f_y^* = -\beta(\frac{\gamma}{\delta}) \tag{2.54}$$

$$g_x^* = \delta(\frac{\alpha}{\beta}) \tag{2.55}$$

$$g_y^* = -\gamma + \delta(\frac{\gamma}{\delta}) = -\gamma + \gamma = 0 \tag{2.56}$$

Hence (from 2.42)

$$\lambda = \pm\sqrt{-\alpha\gamma} \tag{2.57}$$

and so $\lambda = -\sqrt{\alpha\gamma}i$ or $\lambda = \sqrt{\alpha\gamma}i$.

Hence

$$\begin{pmatrix}\epsilon\\ \delta\end{pmatrix} = c_1\boldsymbol{v_1}e^{\sqrt{\alpha\gamma}i} + c_2\boldsymbol{v_2}e^{-\sqrt{\alpha\gamma}i}. \tag{2.58}$$

$$= c_1\boldsymbol{v_1}(cos\sqrt{\alpha\gamma} + isin\sqrt{\alpha\gamma}) + c_2\boldsymbol{v_2}(cos(-\sqrt{\alpha\gamma}) + isin(-\sqrt{\alpha\gamma})) \tag{2.59}$$

$$= \boldsymbol{a}cos\sqrt{\alpha\gamma} + i\boldsymbol{b}sin\sqrt{\alpha\gamma} \tag{2.60}$$

where

$$\boldsymbol{a} = c_1\boldsymbol{v_1} + c_2\boldsymbol{v_2} \text{ and} \tag{2.61}$$

$$\boldsymbol{b} = c_1\boldsymbol{v_1} - c_2\boldsymbol{v_2} \tag{2.62}$$

There is no growth or decay and so we conclude that the fixed point is a centre. The perturbation oscillates near this fixed point. The angular frequency of the oscillations is $\omega = \sqrt{\alpha\gamma}$ and so the period of an oscillation is $T = \frac{2\pi}{\sqrt{\alpha\gamma}}$.

2.2.2 Conserved quantity

If we remove the dependence on time from our equations, we find that there is a quantity that is constant irrespective of time.

From our equations 2.18, we have that

$$\frac{dt}{dx} = \frac{1}{\alpha x - \beta xy} \text{ so} \tag{2.63}$$

$$\frac{dy}{dx} = \frac{dy}{dt}\frac{dt}{dx} = \frac{-\gamma y + \delta xy}{\alpha x - \beta xy} \tag{2.64}$$

which can be rearranged into

$$\int \frac{\alpha - \beta y}{y} dy = \int \frac{-\gamma}{x} + \delta dx \tag{2.65}$$

$$\implies \alpha \cdot lny - \beta y = -\gamma \cdot lnx + \delta x + C(x, y) \tag{2.66}$$

$$\implies C(x, y) = \alpha \cdot lny - \beta y + \gamma \cdot lnx - \delta x. \tag{2.67}$$

This function C does not depend on time which we show by differentiating by t:

$$\frac{dC}{dt} = \frac{\partial C}{\partial x}\frac{dx}{dt} + \frac{\partial C}{\partial y}\frac{dy}{dt} \tag{2.68}$$

and using

$$\frac{\partial C}{\partial x} = \frac{\gamma}{x} - \delta \qquad \frac{\partial C}{\partial y} = \frac{\alpha}{y} - \beta \tag{2.69}$$

$$\frac{dx}{dt} = \alpha x - \beta xy \qquad \frac{dy}{dt} = -\gamma y + \delta xy \tag{2.70}$$

we have

$$\frac{dC}{dt} = (\frac{\gamma}{x} - \delta)(\alpha x - \beta xy) + (\frac{\alpha}{y} - \beta)(-\gamma y + \delta xy) \tag{2.71}$$

$$= 0. \tag{2.72}$$

We call this C a conserved quantity as it doesn't change over time.

2.3 An Example

Let us introduce an example prey and predator species and consider their population change over 500 years, using one year as the unit of time. The prey reproduces on average every five years, so our parameter $\alpha = \frac{1}{5}$. A predator dies on average every 20 years, so $\gamma = \frac{1}{20}$. It will kill a prey when it meets one 1 in 4 times on average, so $\beta_1 = \frac{1}{4}$. We let the rate that a given predator-prey pair will meet in a year be $\rho = \frac{1}{100}$.

We assume that the predator is four times as large as the prey, and we use the 10% rule of energy transfer, meaning a predator must eat roughly 10kg of prey biomass to produce 1kg of predator biomass. Therefore, we say that to have enough biomass to produce another predator, one must eat, on average, 40 prey, so $\sigma = 40$.

Using 2.6, we have $\beta = \frac{1}{4} \cdot \frac{1}{100} = \frac{1}{400}$. Using 2.9, we have $\delta = \frac{1}{40} \cdot \frac{1}{4} \cdot \frac{1}{100} = \frac{1}{16000}$.

Our equations are now:

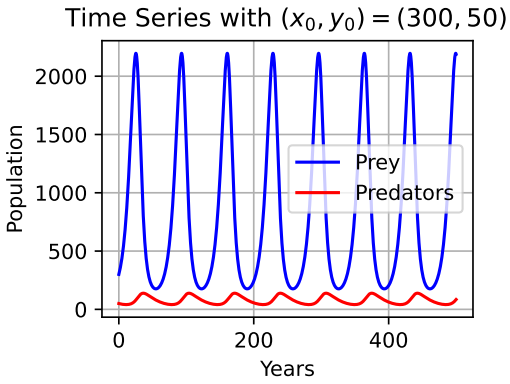
$$\dot{x} = \frac{1}{5} \cdot x - \frac{1}{400} \cdot xy \text{ and} \tag{2.73}$$

$$\dot{y} = -\frac{1}{40} \cdot y + \frac{1}{16000} \cdot xy. \tag{2.74}$$

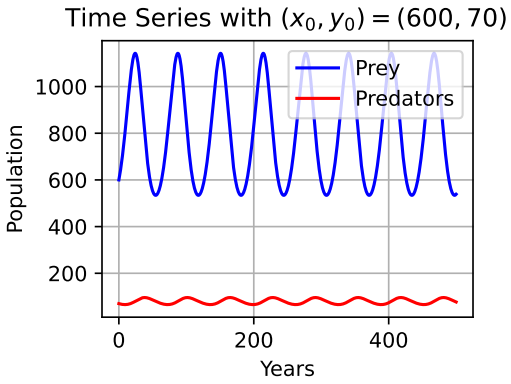
Let us set $x_0 = 300$ and $y_0 = 50$. We solve this initial value problem numerically using the scipy function `solve_ivp`. The result is displayed in figure 2.1a. We see that both the prey and the predators' populations oscillate with a wavelength of approximately 67 years. The difference between the maximum population of prey and the minimum (the prey line's wave height) is approximately 2000 which is much larger than the minimum value of 250 prey. Using 2.21, the non-zero fixed point for the equations is $(\frac{1}{20} \cdot 16000, \frac{1}{5} \cdot 400) = (800, 80)$.

If we set the initial values closer to this fixed point, e.g. $x_0 = 600$ and $y_0 = 70$, we can reduce the difference between the maximum and minimum values. The results for these initial values are displayed in figure 2.1b. In this scenario, the prey line's wave height is reduced to 600 and its wavelength is approximately 63 years. To better see how the two populations change in relation to each other, we display this on a twin-axis grid in figure 2.1c (and plot from $t = 0$ to $t = 200$ for a clearer picture). We can see that the predator's oscillation happens approximately 10 years ahead of the prey's, but apart from that they follow the same pattern (ignoring magnitude).

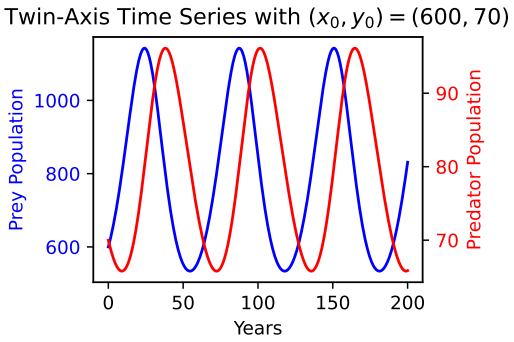
In reality, we can think of this as a cycle: prey population increases because of a low predator count, causing the predator population to increase because there are more prey to eat, causing the prey count to reduce, causing predators to starve, and the cycle starts over again. By plotting predators against prey in a phase portrait (figure 2.1d), we see that these cycles repeat exactly. This is because of the conserved quantity discussed earlier.



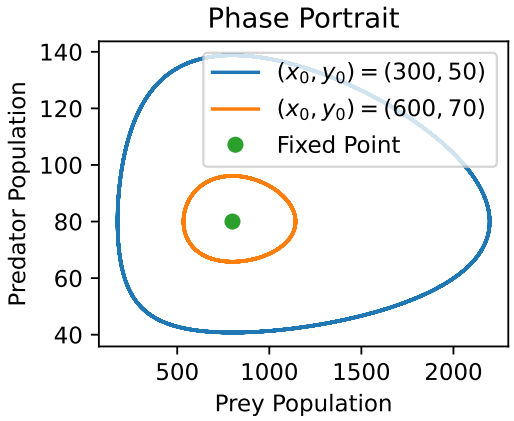
(a) $(x_0, y_0) = (300, 50)$



(b) $(x_0, y_0) = (600, 70)$



(c) Twin-Axes, $(x_0, y_0) = (600, 70)$



(d) Phase Portrait

Figure 2.1: Solutions to the Lotka-Volterra example.

Chapter 3

Discretising the Lotka-Volterra Model

In order to build up towards our individual-based model of predator-prey interactions, we will start by discretising the Lotka-Volterra model that we outlined above.

3.1 Discretising Time

To simplify our eventual model, we will want to think about time in discrete steps rather than a continuous timeline. This will allow us to chain events and run the simulation on a computer which works through these events sequentially.

We introduce the idea of a turn which will be our discrete unit of time. We will have multiple events that happen within a turn, and after these events have occurred, the next turn happens. So our simulation becomes a sequence of events, the turn is just a way of conceptualising 'batches' of these events occurring.

To create a model with discrete time, we will convert our equations into sequence of events. Let us think about what is happening in our equations:

$$\dot{x} = \alpha x - \beta xy \text{ and} \tag{3.1}$$
$$\dot{y} = -\gamma y + \delta xy. \tag{3.2}$$

Firstly let us consider how x changes over time. The change in x, $\dot{x}$, is equal to a reproduction term and a being eaten term. We can convert this to discrete time by saying that every turn, x will increase by some reproduction quantity and decrease by some death quantity.

3.1.1 Defining the Events

We define the effect on x from reproduction as

$$x \leftarrow x + \alpha_{\text{dt}} x \tag{3.3}$$

where α_{dt} is the reproduction rate of the prey per turn. We will use the subscript 'dt' to reference parameters that relate to our discrete time model.

We next define the effect on x from dying by being eaten as

$$x \leftarrow x - \beta_{\text{dt}} xy \tag{3.4}$$

where β_{dt} is the chance that a unique predator will eat a unique prey in a turn.

To design how y should change, we will firstly decompose δ into $\delta = \frac{1}{\sigma}\beta$ using equation 2.9. This makes the y equation:

$$\dot{y} = -\gamma y + \frac{1}{\sigma}\beta xy. \tag{3.5}$$

The predator population decreases from spontaneous death, and increases by gaining biomass from eating prey and reproducing. Our decomposition of δ has shown that the number of predators born is the number of prey eaten multiplied by $1/\sigma$. Firstly we define the effect on y from dying in a turn as:

$$y \leftarrow y - \gamma_{\text{dt}} y \tag{3.6}$$

where γ_{dt} is the predator's death rate per turn. We will then define the effect on y from reproduction as:

$$y \leftarrow y + \frac{1}{\sigma_{\text{dt}}} \cdot (\beta_{\text{dt}} xy) \tag{3.7}$$

where σ_{dt} is the average number of prey a predator needs to eat to reproduce per turn.

This method of discretisation is similar to Euler's Method [4], where a continuous function $f(x) = x$ is discretised by increasing the independent variable by steps of h, so two consecutive points would be of distance $h = x_{i+1} - x_i$, and we would use the value of x_i to estimate x_{i+1} by $x_{i+1} = x_i + h \cdot f'(x)$. In our case the h is a turn which is one unit. However we diverge away from Euler's Method by having these events occur in a sequence and updating the values one-by-one, rather than calculating the the change in the populations and only changing the populations at the end. We change the values separately to ease the conversion to an individual-based model. An individual cannot be being eaten and be reproducing at the same time for example.

We will see that our discretisation model diverges from the continuous model in the same way that Euler's Method accumulates errors.

3.1.2 Constructing the Model

Now that we have defined the four events that we wish to occur each turn, we next decide the order in which they occur within the turn. It is natural that we have predator reproduction happening after prey death (since the predators are using the prey biomass to reproduce). Apart from that, it does not matter too much the order of events since turns will happen one after another the order of an event within the turn does not make much of a difference over a longer time period.

We define the turn as: prey eaten $\rightarrow$ predator reproduction $\rightarrow$ prey spawn $\rightarrow$ predator death.

It is possible that in the prey eating event, $x \leq \beta_{\text{dt}} xy$. If this happens, we set $x = 0$ and we say that the prey are extinct. This is a way that our discrete model differs from the continuous one as extinction cannot occur in the Lotka-Volterra model (with non-trivial parameters and initial populations).

We encode this discrete model into python in listing B.1.

3.1.3 Testing it using the Example

We want to translate the example we conducted with the Lotka-Volterra model in subsection 2.3. We first need to choose what a turn represents. We will say that a turn represents a month, as if a turn represented a year the discrete time intervals would be too large to adequately resemble real life. This means that for any rate dependent on time we used in the example for Lotka-Volterra, we will want to divide them by 12, since we used a year as the unit of time when time was continuous. Our parameters become:

$$\alpha_{\text{dt}} = \alpha \cdot \frac{1}{12} = \frac{1}{5 \cdot 12} = 0.01667, \tag{3.8}$$

$$\beta_{\text{dt}} = \beta \cdot \frac{1}{12} = \frac{1}{400 \cdot 12} = 0.0002833, \tag{3.9}$$

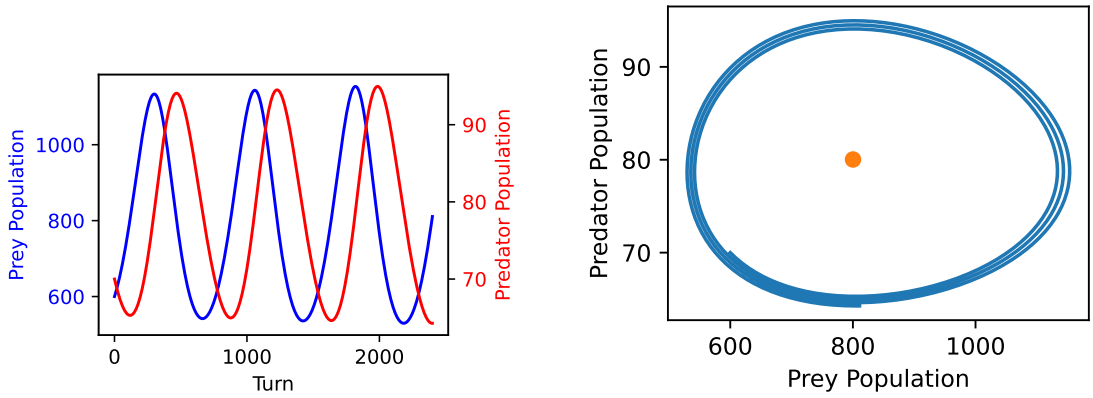
$$\gamma_{\text{dt}} = \gamma \cdot \frac{1}{12} = \frac{1}{20 \cdot 12} = 0.004167, \text{ and} \tag{3.10}$$

$$\frac{1}{\sigma_{\text{dt}}} = \frac{1}{\sigma} = \frac{1}{40} = 0.025 \tag{3.11}$$

all to four significant figures. Let us run the simulation for 500 years, so 6000 turns. We plot the first 2400 turns in figure 3.1.

Our results are very similar to the ones from our Lotka-Volterra run. However, we see that in the phase portrait, figure 3.1b, the spiral is not fixed to one path, as we no longer have the conserved quantity that we did with the Lotka-Volterra equations. This means that it is possible for the model to spiral out to extinction.

When we plot the prey population from the discrete time results with those from Lotka-Volterra, we see how similar the results are, as shown in figure 3.2.



(a) Twin-Axis Time Series

(b) Phase Portrait

Figure 3.1: Solution to our example using the discrete time model.

The only difference we see is that the discrete time model's wave heights are slowly increasing, which is what we could see happening with the spiralling out in the phase portrait.

We can understand this spiralling out by considering what is happening when we go from continuous to discrete time. The variables lose some of their 'reactivity'. Consider when prey is increasing: in the Lotka-Volterra model, the predator reproduction rate starts increasing straight away, and more predators means that prey increase slows down, whereas in a discrete model, there is a lag between when the prey increases, and when the predators get the benefit of that increase, allowing the prey population to grow with less diminishment. This also occurs on the other end, inwhich as the growing predator population, spurred on by the larger prey population causes a larger dip in the prey population. The issue with the model is that there is nothing limiting this, and that over time these effects compound which is why we see the spiralling out.

This mechanism becomes clear to us when we plot the discrete time model with a year representing a turn, and the parameters scaled accordingly. In figure 3.3, we see how the wave heights increase dramatically, and in the phase portrait in figure 3.4 we see how the populations spiral out away from the fixed point.

We have shown that we will get results more closely reflecting the Lotka-Volterra model if we reduce the time interval (what a turn represents). This is not surprising as when we reduce the time interval we are getting 'closer' to a continuous model. A continuous model has infinitesimally small time steps. We can again compare our results with Euler's Method, where smaller steps reduce the error in the numerical approximation.

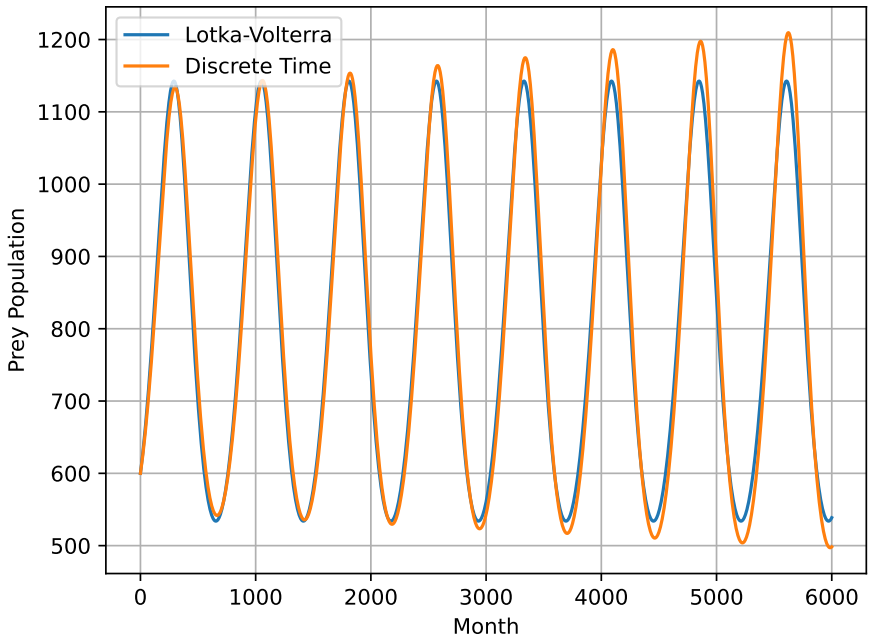


Figure 3.2: Time Series showing the prey population dynamics of the results from the Lotka-Volterra and discrete time models.

3.2 Discretising Animals

Our next step towards our individual-based model is to turn animal populations from continuous to discrete values.

A naive approach would be to have an event occur, and then round the resulting population number to the nearest digit. The issue that arises is that if the population change from an event has magnitude under 0.5, the population can then become stuck at a value. If this is a case for every event, then the simulation ceases to be able to change the animal values.

A better way to handle the rounding is to use stochastic rounding [2]. This is where the probability that it is rounded up is the same as the decimal part of the number, otherwise it is rounded down. We can write this out mathematically:

$$x_{\text{rounded}} = \begin{cases} \lceil x \rceil & \text{with probability } x - \lfloor x \rfloor \\ \lfloor x \rfloor & \text{else.} \end{cases} \tag{3.12}$$

This is useful because it preserves the meaning of the decimal part of the number, as in it is more likely to be rounded up if the number is nearly equal to that number anyway, while also allowing populations to change even if the population change values have magnitude less than 0.5. An implementation in Python can be seen

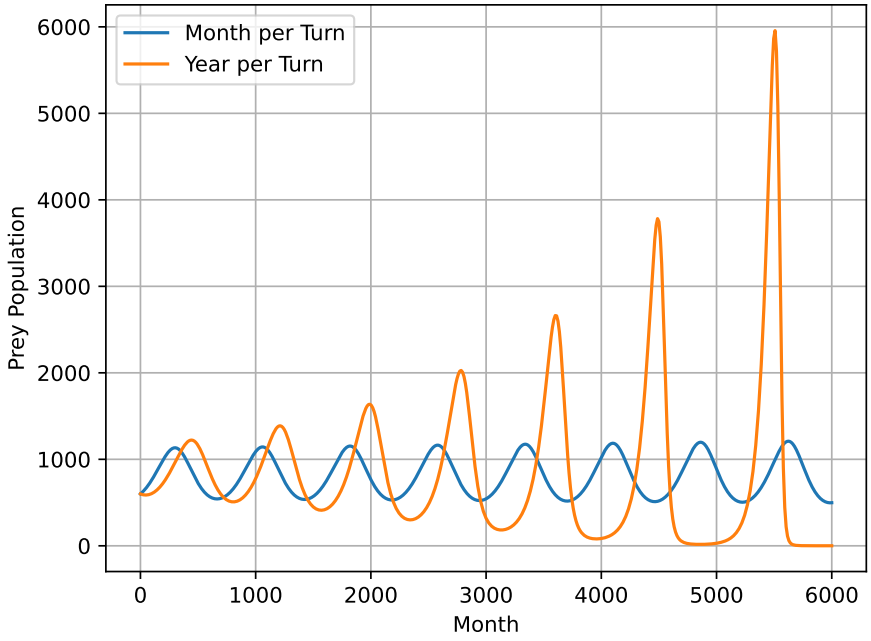


Figure 3.3: Time Series showing the prey population dynamics for the discrete time model when a turn models a month versus a year.

in listing B.2.

However, it does introduce probability into our model, meaning that our outcomes are no longer deterministic. This makes it more difficult to take conclusions from it because we do not know if a run is representative of others. This is why we will run these stochastic models multiple times to get an understanding of the overall behaviour.

A behaviour that discretising the animal populations creates is the possibility of predator extinction as well as prey. In the case of predator extinction, we stop the simulation, because without predators around the prey's population grows exponentially.

3.2.1 Implementation and Results

We implement this discrete time-and-animals model in python, which can be seen in listing B.3. We run this model with the same parameters that we used in our discrete animal model, with a turn representing a month. The results can be viewed in figure 3.5. In the predator-prey time series, we can see the stochasticity in effect. The wave height increases from the first to the second wave cycle but then decreases for the third.

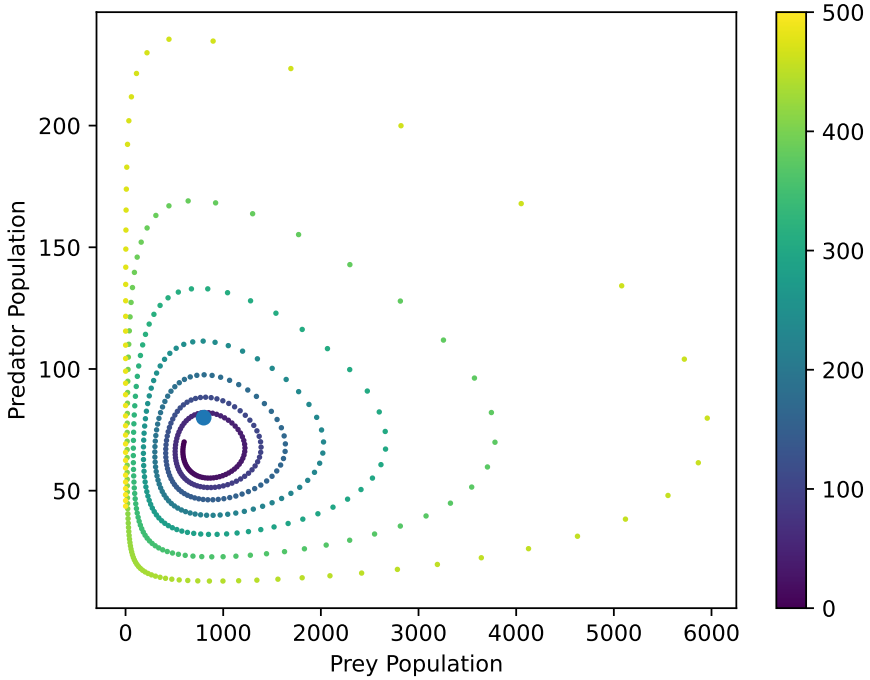
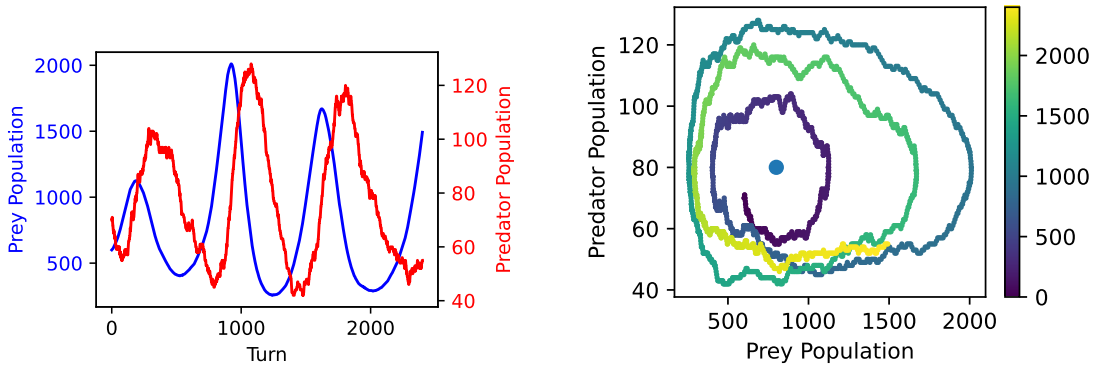


Figure 3.4: Phase portrait showing the population dynamics for the discrete time model when a turn models a year.

When we plot the result for more turns and compare it to the discrete time model results in figure 3.6a, we find that it has a tendency to spiral out. In the plot with three runs, figure 3.6b, we see that the stochasticity that we introduced with the stochastic rounding has made the simulation very non-deterministic. Two of the simulations spiral outwards while one seems to be more close to our discrete time results, spiralling out less quickly or not at all.

These differences in results can be explained by the variance caused by the stochastic rounding. The reason why this noise greater spiralling out on average, is that the noise is more likely to move the populations away from the center rather than towards it. The easiest way to think about this is to consider the phase portrait in figure 3.5b. The orbits are roughly an oval. If we pick a point on one of the orbits and we were to choose a random direction to walk, we would most likely move outwards because that gives us the largest angle for our random direction to fall in. This tendency to move out of the orbit over time pushes the spiral outwards. We can see that this effect is a lot larger than the spiralling out effect introduced when we discretised time.

As probability is only introduced to deal with rounding, we would expect its effects to be more significant on smaller populations.



(a) Twin-Axis Time Series

(b) Phase Portrait

Figure 3.5: Solution to our example using the discrete time and animals model.

3.3 Treating the Animals Individually

The next step in converting to an individual-based model is to have the events happen to every animal individually rather than enacting the events on the whole population at once. In the prey reproduction event, every prey either reproduces or they do not. In the predator death event the same idea.

For prey being eaten, β_{si}, si standing for simple IBM, is the rate that an an individual predator will eat an individual prey. When solving this for an individual pair, we are forced to stochastically round to either 0 (not eaten) or one (eaten). In this way every interaction has to be modelled stochastically, because it is always either 0 or 1. This means that we can model every interaction as a Bernoulli trial. On the event scale, we can generalise to binomial functions to decide what happens.

We define our new events as:

$$-\Delta x_{\text{death}} \sim \text{Bin}(x \cdot y, \beta_{\text{si}}) \text{ and} \tag{3.13}$$

$$x \leftarrow \max\{x + \Delta x_{\text{death}}, 0\}, \tag{3.14}$$

$$\Delta y_{\text{reproduction}} \sim \text{Bin}(-\Delta x_{\text{death}}, \frac{1}{\sigma_{\text{si}}}) \text{ and} \tag{3.15}$$

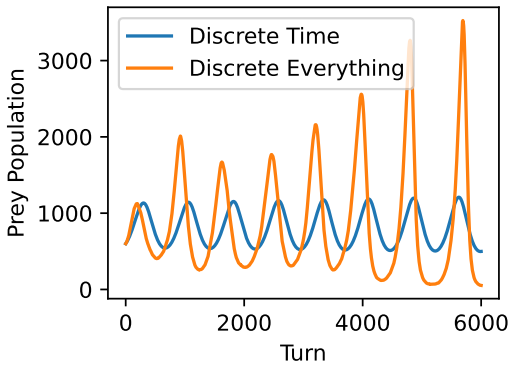
$$y \leftarrow y + y_{\text{reproduction}}, \tag{3.16}$$

$$\Delta x_{\text{reproduction}} \sim \text{Bin}(x, \alpha_{\text{si}}) \text{ and} \tag{3.17}$$

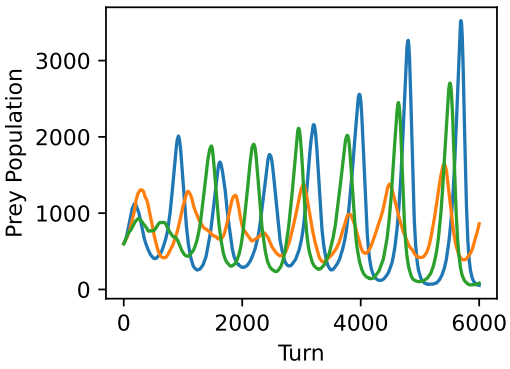
$$x \leftarrow x + x_{\text{reproduction}}, \text{ then} \tag{3.18}$$

$$-\Delta y_{\text{death}} \sim \text{Bin}(x, \gamma_{\text{si}}) \text{ and} \tag{3.19}$$

$$y \leftarrow \max\{y + \Delta y_{\text{death}}, 0\}. \tag{3.20}$$



(a) Discrete Time vs Discrete Everything

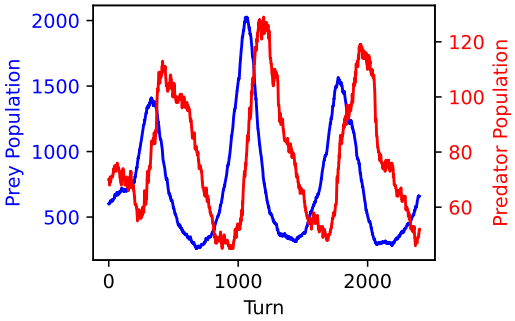


(b) Three runs of the model

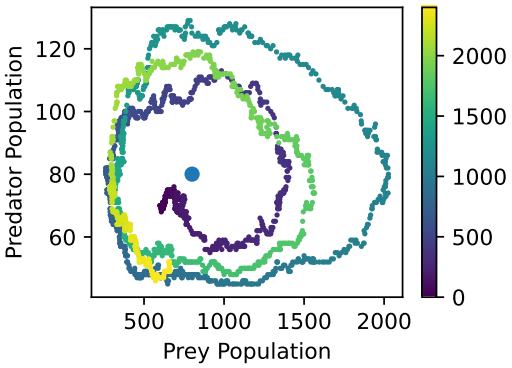
Figure 3.6: Comparing the prey population results for the discrete time-and-animals model.

3.3.1 Implementation and Results

We implement this in python in listing B.4 and plot the results of the simulation with same parameters again in figure 3.7. The time series is very similar to the one from our discrete time-and-animals model. However, the phase portrait appears to be noisier than than from the model before. This is what we would expect since the model now relies on much more probabilistic events, adding greater variance to our results.



(a) Twin-Axis Time Series



(b) Phase Portrait

Figure 3.7: Solution to our example using the simple individual-based model.

In figure 3.8, we plot three runs of the simple IBM model. We can see that the waves are noisy, yet we do not see a high amount of spiralling out.

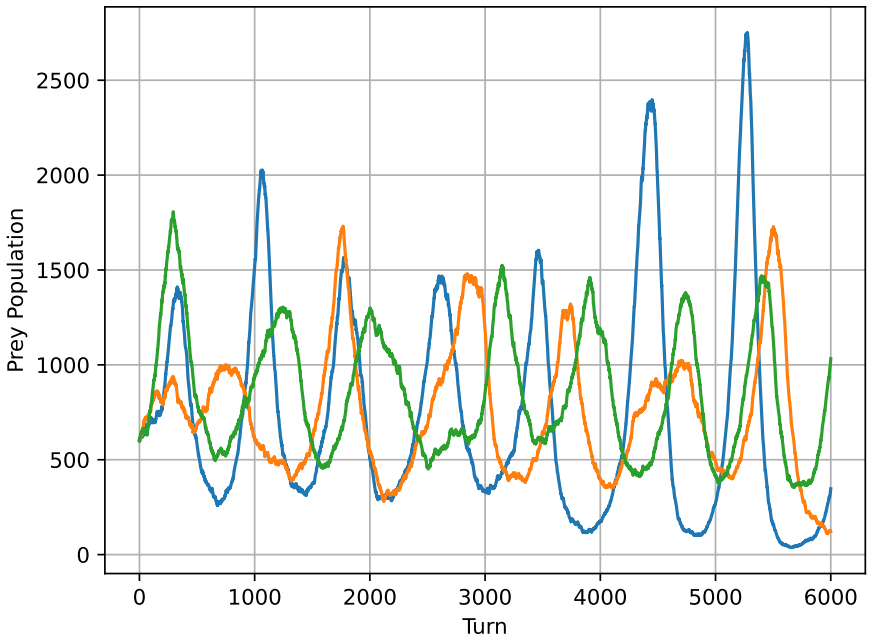


Figure 3.8: Time Series showing the prey population dynamics of three runs of the simple individual-based model.

In figure 3.9, we plt the four models together. This makes clear the similarities between the two deterministic models, and the similaries between the two stochastic ones, as well as the large difference between these two groups.

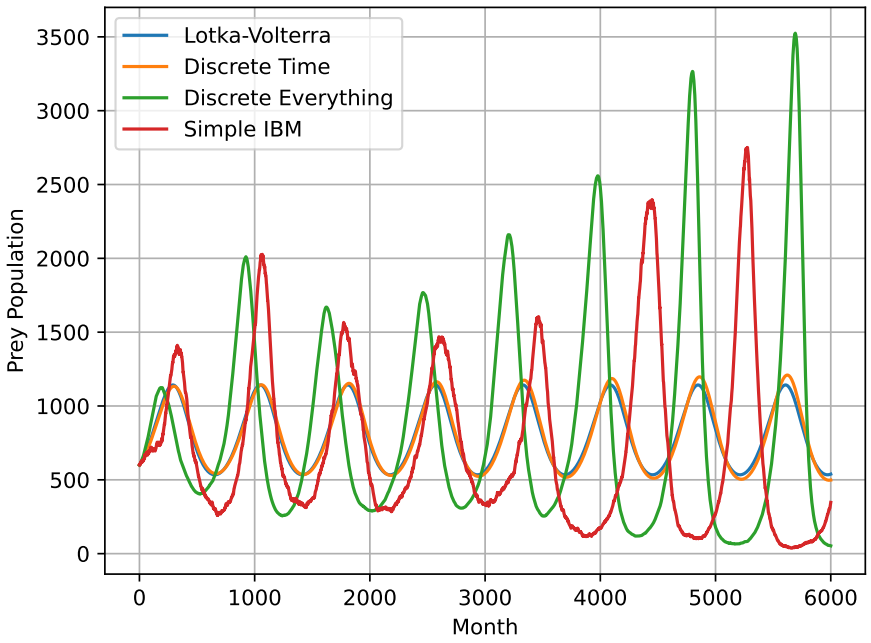


Figure 3.9: Time Series showing the prey population dynamics of the four solvers.

Chapter 4

Designing Our Individual-Based Model

Our models that we have created and analysed so far have taken steps away from the continuous Lotka-Volterra model and towards an individual-based model. We will take the next and final step by adding a spatial element to our model. We define a grid in which the animals are able to move around in. We will discuss the many directions that the grid design can go in, look at how this model disagrees with the assumption that the animals are well-mixed, and analyse how our results compare with those from the continuous model.

4.1 Defining the Grid

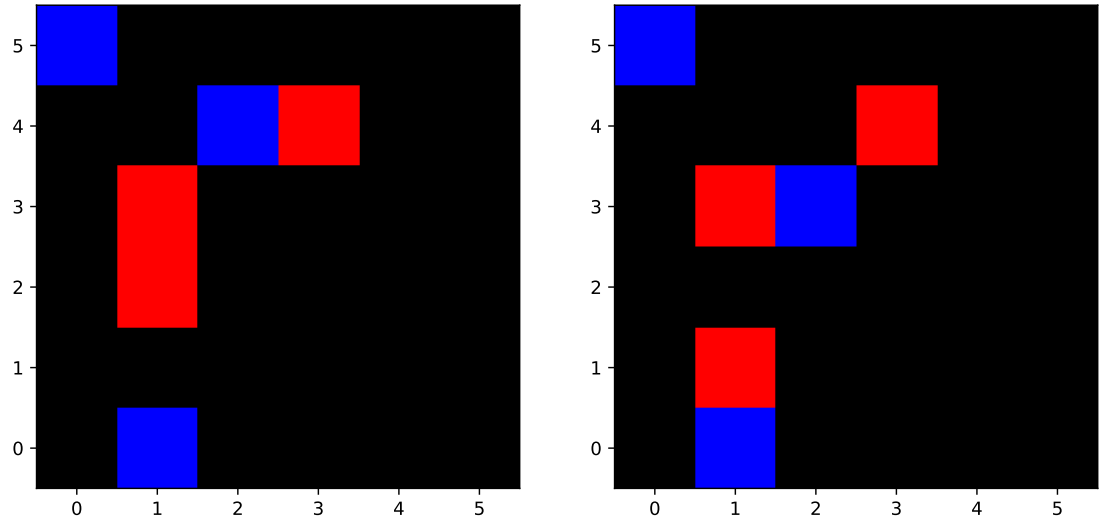
4.2 The Habitat

We define the grid to be rectangular with m rows and n columns. To start with, x_0 prey and y_0 predators are randomly place onto the grid. For example, in figure 4.1a we place three predators and three prey on a six-by-six grid. To display the grid we colour points containing prey blue and ones with predators red (and purple if both). When there is a large number of animals compared to the grid size, we use the strength of the colours to represent the size of the populations in each grid.

4.2.1 Movement

We will allow an animal to move one square up, right, left, or down on the grid, or not move at all, every turn. (We add standing still as an option to stop animals from becoming out of phase with each other.) We assign the probability of not moving as 0.5, and the probability of moving any particular direction as 0.125. In

figure 4.1b, one turn has taken place. We see that a predator moved from $(1, 2)$ down to $(1, 1)$ and a prey moved from $(2, 4)$ down to $(2, 3)$.



(a) Initial setup.

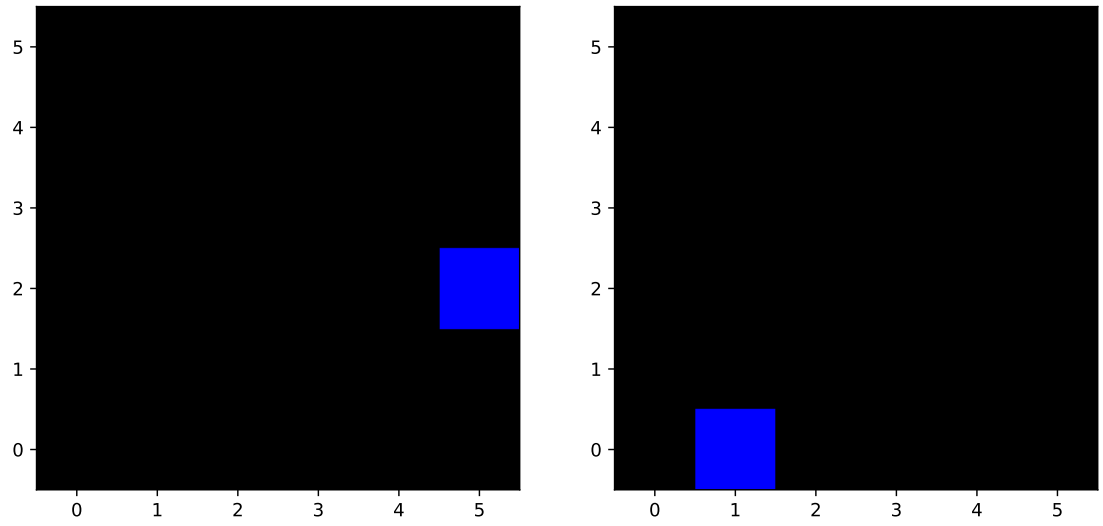
(b) Grid after one turn.

Figure 4.1: Grids showing the locations of animals with prey in blue and predators in red.

To fully define movement, we must decide what happens to an animal located on the edge of the grid that 'decides' to move off it. We will refer to this as 'escaping'. For example, if the prey in position $(0, 5)$ in figure 4.1a elects to move up or left.

We could handle this by not allowing escape at all. We will refer to this as 'no escape' movement. Another option is that if an animal escapes, it is readded at a random point on the edge of the grid. We display an example of this in figure 4.2; the prey is at $(5, 2)$ and on the next turn it walks off the grid (moves rightwards) and so gets transported to $(1, 0)$. We will refer to this as 'random edge point' movement. This mechanic can be conceptualised as simulating a rough form of immigration and emmigration. It is as if one animal has emmigrated at one point and another at the same time has immigrated at a different place. This is helpful if we want the grid to represent a subsection of a wider habitat.

We expect our choice of movement to matter more in smaller grids and over longer time periods. On a large grid, it would take time for the effects of edge movement to diffuse into the centre.



(a) Before

(b) After

Figure 4.2: Grids showing a prey walk off the edge of the grid and be moved to a random point on the edge.

4.3 Translating our Earlier Events

4.3.1 Eating

We will give the predators and opportunity to eat prey only when they land on the same grid point. When a prey lands on the same point as a predator, there will be a chance $\beta_{1\text{sim}}$ that the prey is eaten, determined probabilistically. We expect this to be analogous to the parameter β_1 which represented the same concept in our Lotka-Volterra derivation earlier.

We had that $\beta = \beta_1 \rho$ where ρ is the meeting rate of a unique predator and prey. We will look at how often predators and prey meet in our model and compare it with this ρ.

We then must decide what happens when multiple prey and predators meet at a grid point. One way to handle this would be to pair every predator with a prey where possible and run the prey-eating predator-reproduction events with each pair. This would mean that if there were g prey and h predators at a grid space with $g < h$, there would be g pairs. However, this would mean that $gh - g$ unique predator-prey pairs would not interact with each other, even though they landed on the same point.

We want the number of pairs interacting to be as close as possible to gh because we will be calculating the chance that two animals of different species meet, which

would be made irrelevant if they were blocked from interacting when they met because there were multiple other animals on that grid point as well.

We decide instead to pair as many predators and prey together as we can. We take a prey and allow each predator to try to eat it until we have either run out of predators or the prey has died. We do this iteratively on every prey. For a single prey, it would have the chance $1 - \beta_{1\text{sim}}$ of surviving the first predator, if it does, it again has a chance $1 - \beta_{1\text{sim}}$ of surviving the next predator and its chance of surviving both is then $(1 - \beta_{1\text{sim}})^2$. Hence the prey has a $(1 - \beta_{1\text{sim}})^h$ chance of surviving all the predators and so its chance of being eaten is $1 - (1 - \beta_{1\text{sim}})^h$. The number of prey eaten would be $(1 - (1 - \beta_{1\text{sim}})^h)g$. This is less than the number of unique pairs times the chance of being eaten $\beta_{1\text{sim}}gh$. This will be an issue when we attempt to compare with the earlier models since $\beta_{1\text{sim}}gh$ is the theoretical number that should be eaten.

4.3.2 Reproduction

We will simulate reproduction asexually, with each animal given the same probability of reproducing. For prey reproduction, each has a chance α_{sim} to reproduce.

We can handle predator reproduction in multiple ways. One option is that once a prey is eaten, there is a chance $1/\sigma_{\text{sim}}$ that the predator reproduces. We refer to this reproduction mechanism as 'dependent on prey death'. Another option is to give a predator a chance to reproduce once it meets a prey regardless of whether or not it eats it. This would simplify the model while also reducing its realism. We refer to this reproduction mechanism as 'independent of prey death' in the code, but we will not discuss this in the report and instead define prey reproduction as dependent on prey death as it naturally follows from our Lotka-Volterra derivation.

Another option would have been to choose to track how many prey a predator has eaten, and spawn a new predator when it eats σ_{sim} prey. In this model we would expect less predators to be produced as predators may die with a non-zero number of prey accumulated. It would make a larger difference if the prey are very sparse and the number that needed to be eaten for reproduction is high, meaning that predators would rarely reach the threshold to reproduce. Arguably this would make the model more realistic as animals would not be able to reproduce if they have not consumed enough biomass to do so. However we will not explore this as it causes animals to no longer be interchangeable, as they would differ by the amount they have eaten, which would increase the complexity of the simulation.

4.3.3 Predator Death

We say that each predator has a chance γ_{sim} that they will die.

4.4 A Turn

We will order a turn as: animals moving → predators eating preys → predators reproducing → preys reproducing → predators dying.

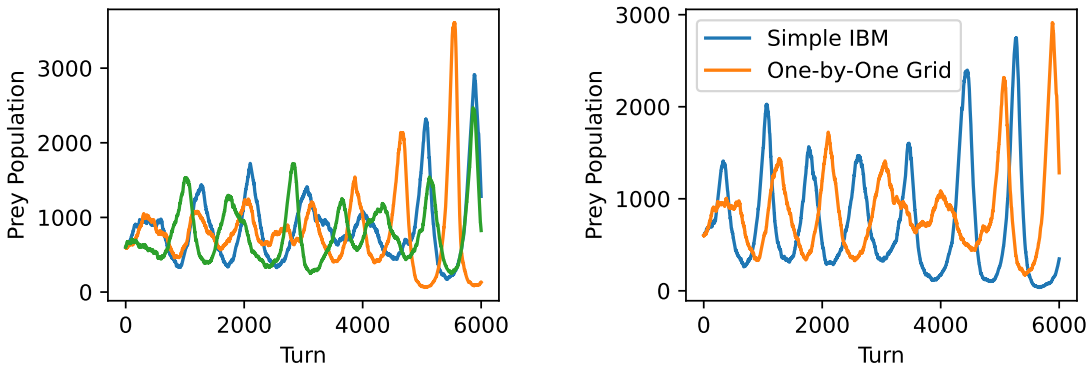
Chapter 5

Analysing the IBM

We implement this model in Python. As it is too large to fit in the appendix, the code has been made available in a github repository: https://github.com/benjchez/Predator-Prey-Population-Dynamics.

5.1 One-by-One Grid

To verify the model, we will investigate the population dynamics in a one-by-one grid. This should be comparable with our simple IBM model. We plot three simulations on figure 5.1a. We can see that the one-by-one grid seems to cause similar increasing wave heights over time. When we plot one of these simulations against a run of the simple IBM, figure 5.1b, we find the two to be very similar. The simple IBM model has the same dynamics as our one-by-one grid.



(a) Three Runs

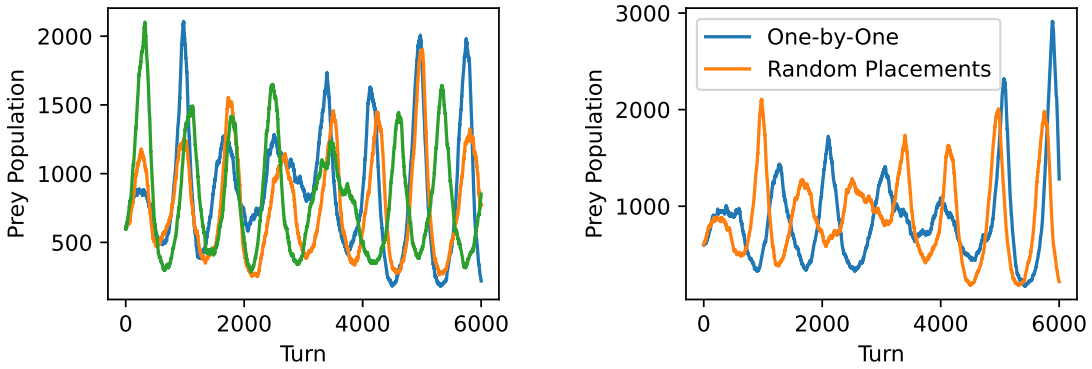
(b) Plotted against simple IBM

Figure 5.1: Plotting the prey population dynamics of the simulation on a one-by-one grid.

5.2 Grid Without Movement

We next want to introduce space. We will do this firstly without movement occurring so that we can isolate the changes. So for this experiment, every turn the animals are placed randomly on the grid, and then the turn ensues without the movement event. We expect the main difference of this model compared to the one-by-one grid to be the effects of the difference between the number of unique preys and predators at a point, and the number which do interact with each other.

We plot three runs of this experiment in figure 5.2a and compare one of them against the one-by-one grid experiment in 5.2b. The runs show less signs of spiralling outward and stay in-phase with other for longer. This suggests that the effect of the predators missing out on chances to eat prey causes a dampening effect. The intuition would be that as the predator population is growing rapidly they start to kill off prey more in the grid points before they all get a chance at eating the prey. This reduces the predator growth and shortens their wave height, combatting the effects of the spiralling out discussed earlier.



(a) Three Runs

(b) Plotted against one-by-one grid

Figure 5.2: Plotting the prey population dynamics when animals are arranged randomly on the grid after every turn and there is no movement event.

5.3 IBM Experiment

Let us now run an experiment with movement defined (using the usual parameters) on a 40-by-30 grid and use the no escape movement mechanic discussed earlier.

We plot the prey and predator populations at each turn in figure 5.3a on a twin-axis grid. When we plot the simulation as a phase portrait: figure 5.3b, we see that the populations spiral inwards.

In figure 5.4, we plot the Lotka-Volterra prey population dynamics against our IBM simulation. The prey and predator lines have larger wave heights in the IBM than in Lotka-Volterra, with the prey's first peak being around 3900 for the IBM compared to around 1150 for Lotka-Volterra.

5.4 Animal Mixing

We propose that an important reason for the difference between our IBM model and the Lotka-Volterra model is how the animals arrange themselves on the grid.

In our Lotka-Volterra derivation, we defined the constant ρ to be the meeting rate of a given predator and prey. Implicit in this, we assumed that the meeting rate does not change over time.

In our individual-based model, we cannot make this assumption. Therefore, We want to investigate how the meeting rate changes in a simulation. On an m-by-n grid, we define the rate of meeting in the simulation at turn t as

$$\rho_{\text{sim}}(t) = \frac{\sum_{i,j=0}^{m,n} x_p(i,j,t) \cdot y_p(i,j,t)}{x(t) \cdot y(t)} \tag{5.1}$$

where x_p and y_p give the numbers of prey and predators at a grid point.

We can find the meeting rate over a time period $T_{k,l} = \{t_k, t_{k+1}, ..., t_l\}$ by taking a weighted average:

$$\rho_{\text{est}}(k,l) = \frac{\sum_{t \in T_{k,l}} \rho_{\text{sim}} \cdot (x(t) \cdot y(t))}{\sum_{t \in T_{k,l}} x(t) \cdot y(t)} \tag{5.2}$$

$$= \frac{\sum_{t \in T_{k,l}} \sum_{i,j=0}^{m,n} x_p(i,j,t) \cdot y_p(i,j,t)}{\sum_{t \in T_{k,l}} x(t) \cdot y(t)}. \tag{5.3}$$

If animals are randomly arranged on the grid, then the chance that one animal meets another on a given turn would be the same as the chance that the second animal is randomly placed on the same grid point as the first. On an m-by-n grid, this would be $\frac{1}{m \cdot n}$. Since the simulation starts by randomly placing the animals on the grid, we expect $\rho_{\text{sim}}(0) \approx \frac{1}{m \cdot n}$. We then want to find out whether the rate of meeting stays at this number, ie $\rho_{\text{est}}(0, l) \approx \frac{1}{m \cdot n}$ for some $l \in \mathbb{N}$.

For this, we should consider how the grid mechanics and the population changes would affect the meeting rate.

5.4.1 Grid Mechanics

Each path that an animal can take from one grid point to another has a probability of happening within a fixed time frame. Since the animal movement mechanics

stays the same throughout the simulation, this probability does not change with the initial time of movement. If we can show that the reverse of each path is equally likely to be taken by an animal at the opposite end of the path, then we can say that the grid mechanics will not unmix the animals.

For a grid with no escape, there is no bias for movement towards any particular point. We can see this by looking at the movement from a point on the edge. Let the grid be 3-by-3, the chance of an animal being in a grid spot be a and the chance of moving in any particular direction be b. There is a probability of b that the animal is at $(0, 2)$. It has a probability of $3a$ that it will move off that spot, as if it tries to move left, it will stay where it is. So the chance of an animal moving away from $(0, 2)$ is $3ab$. There is a probability of $3b$ that an animal is in any of the neighbouring points. In each of the points, it would have a probability of a of moving to $(0, 2)$. Hence the chance of an animal moving into $(0, 2)$ is $3ab$. Therefore there is the same chance of an animal moving out of as of moving into the grid point. The same line of reasoning can be followed for any grid point, proving that this movement mechanic would not break the randomness of the grid.

For random edge point movement, there is a problem. An animal in the the corner of the grid has $2a$ chance of escape. For example an animal in the top left can escape by going up or left. However, it has the same chance that an animal will be moved to it as any other edge on the grid. A non-corner edge has a chance of escape. This means that there is net movement from corners to non-corner edges. This will slightly reduce how well-mixed the animals are in their habitat. To fix this, we make it two times more likely to land in a corner after walking off the edge than in a non-edge corner.

These results show that the grid mechanics alone will not significantly affect ρ. To verify this, we run simulations with no change in population size for 100 turns using the two different grid mechanics first on a 5-by-5 grid, and then on a 10-by-100 grid. We set the predator and prey populations to both be 1000. In figure 5.5, we find that our measure for a well-mixed grid, $\rho_{\text{sim}}(t)$, is close to $1/(m \times n)$ throughout the 100 turns in every simulation. For the 10-by-100 grids, 5.5b and 5.5d, $\rho_{\text{sim}}(t) \approx \frac{1}{10 \cdot 100} = 0.001$ for all time. For the 5-by-5 grids, 5.5a and 5.5c, $\rho_{\text{sim}}(t) \approx \frac{1}{5 \cdot 5} = 0.04$. This is what we expected.

We have shown that the different grid mechanics and different grid shapes will not change how well-mixed the grid is alone. To further verify this, we plot 10 simulations for each scenario for 1000 turns. Figure 5.6 shows how $\rho_{\text{est}}(0, t)$ converges to the values outlined above.

5.4.2 How Events Modify Animal Arrangement

In the creation of our simulation, we assumed that $\rho_{\text{sim}}(t)$ would be approximately $1/(m \times n)$ over time. When we plot $\rho_{\text{sim}}(t)$ in figure 5.7a, we find that it stays

below our expected value for most of the simulation. When we plot $\rho_{\text{est}}(0,t)$ in figure 5.7b, we see that the weighted average drops dramatically, and levels out at around 0.0004. We find that $\rho_{\text{est}}(0,t) = 0.000412$ (3 sf), which is almost exactly half of $1/(m \times n) = 0.000833$ (3 sf).

This is not too surprising. We would expect prey in areas with less predators to live for longer and produce more offspring, causing 'patches' with large numbers of prey. These patches would only exist for as long as it would take for more predators to diffuse into that area, eat prey, and reproduce themselves. The effect of these patches means that a predator is less likely to meet a prey on a turn then if they were both placed on the grid randomly. In figure 5.8, we see in the space of around 300 turns, the absence of many prey in the upper area (5.8a), causing prey to thrive there (5.8b), leading to the growth of the predator population there in 5.8c and 5.8d.

Next we want to answer whether $\rho_{\text{est}}(0,t)$ is predictable. To do this we run the simulation 10 times. First let us plot their prey populations. Figure 5.9a shows the change in prey population of 10 simulations and 5.9b shows this for 10 simulations that had the same grid layout to start with (animals are randomly placed onto one grid, and this is used as the starting grid for 10 simulations). We can see that in both cases, the simulations' prey populations' waves are similar for the first two, and then get more out of phase as the simulations go on. Interestingly, the simulations that start with the same grid do not seem much more correlated than the simulations that did not. We would expect them to be since the simulations start more aligned. This leads to the conclusion that the placement of animals on the grid is less important than the underlying grid mechanics in guiding the outcome of the simulation.

To investigate the 10 simulations with different initial grids further, we plot their $\rho_{\text{est}}(0,t)$ values together in figure 5.10. Interestingly, the values from the different runs all seem to converge to a similar value around 0.0004. We can find the average of $\rho_{\text{est}}(0,6000)$ for the 10 runs as 0.000404 (3sf). Is it coincedental that this value is approximately half $1/(m \times n) = 0.000833$ (3sf)?

What we can conclude is that the $\rho_{\text{est}}(0,t)$ value is determined by the simulation parameters and not the starting arrangement of the animals. The mechanics of the simulation will always arrange the animals in a way that gives this $\rho_{\text{est}}(0,6000)$ value.

5.4.3 Edge Movement Mechanic

To dive further into how this $\rho_{\text{est}}(0,6000)$ value is derived, we can see how it changes with a different movement mechanic and different sized graphs. Our random edge point movement places animals that have escaped the graph at a random point on the edge. We would expect this to promote a better mixing of animals

as if there is a large collection of one species on one side of the map, it can spread them into other areas. Therefore, we expect simulations with random edge point movement to have higher $\rho_{\text{est}}(0, 6000)$ values, closer to the well mixed theoretical value.

We can see that our intuition is correct when we plot $\rho_{\text{est}}(0, t)$ for 10 experiments using random edge point escape, 5.11. The average $\rho_{\text{est}}(0, 6000)$ value is 0.000456 (3sf) showing slightly better mixing. This leads to the conclusion that that the edge movement has a small effect on how the animals mix.

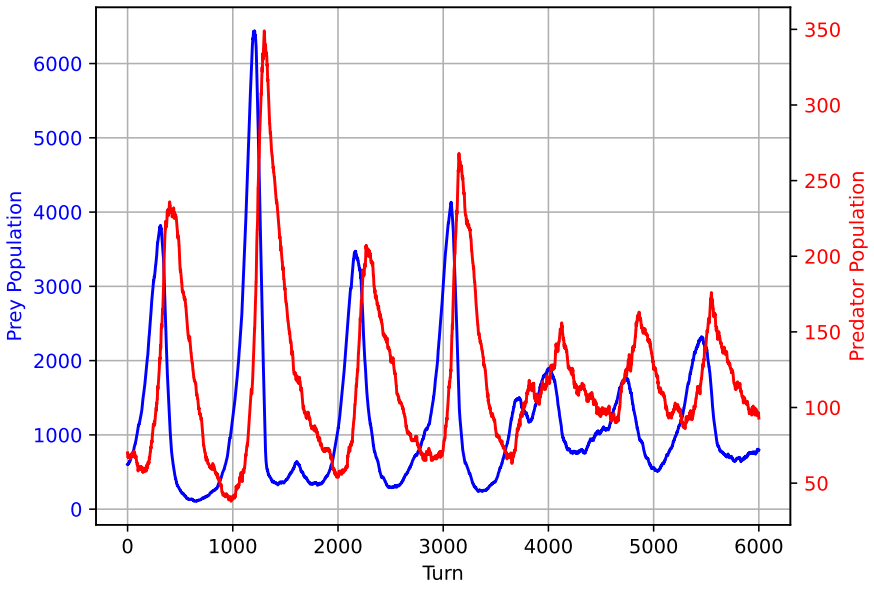
5.4.4 Choice of Unit of Time

For our example, we arbitrarily chose a turn to mean a month. If we instead choose a turn to mean 6 months, will this affect our relationship between ρ_{sim} and ρ? It would mean that animals move less per year, and so we would expect there to be more bunching, causing less mixing of the two species, giving us lower ρ_{sim} when compared to our theoretical ρ.

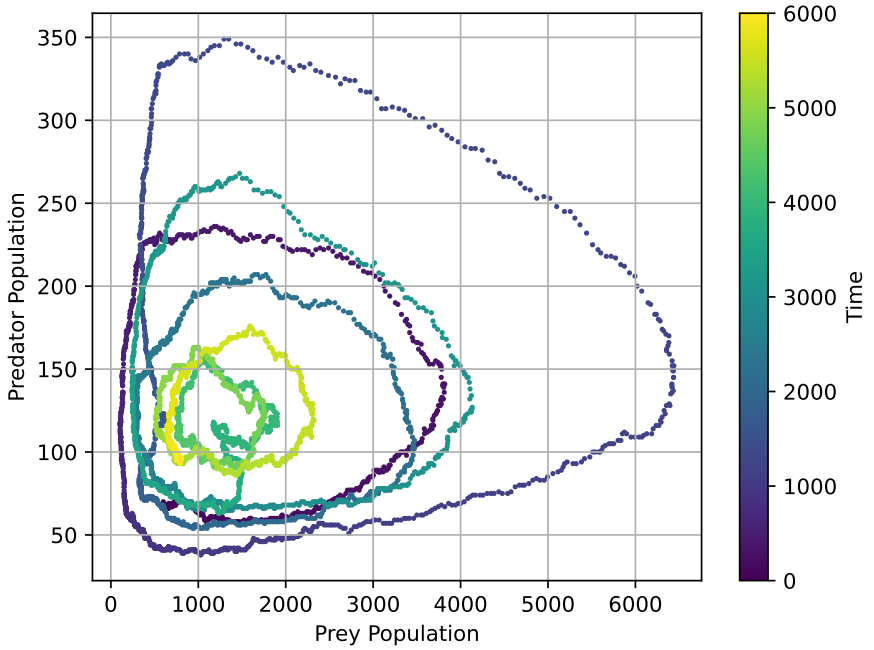
For this experiment, $\alpha_{\text{sim}} = \alpha/2 = 1/10$, $\gamma_{\text{sim}} = \gamma/2 = 1/40$, $\beta_{1\text{sim}} = \beta_1 = 1/4$, and $\sigma_{\text{sim}} = \sigma = 40$. We want $1/(m \times n) = \rho/2 = 1/200$ so we choose $m = 20$ and $n = 10$. We will again run for 500 years so 1000 turns.

In figure 5.12 we plot the $\rho_{\text{est}}(0, t)$ values for 10 simulations. Taking the average of the $\rho_{\text{est}}(0, 1000)$ values, we get 0.00281 (3sf). As our theoretical $\rho = 1/200 = 0.005$, our average is again around half the theoretical value, showing an interesting connection between the theoretical and practical values.

To further examine the connection between simulations with one month per turn and six months per turn, we plot in figure 5.13 one simulation for each scenario on the same grid, with months on the x-axis (so the six months per turn x-values have been scaled up by six). We find that their wavelengths and wave heights are similar, showing that they are in reality simulating a similar reality, only with one taking larger steps through time than the other.



(a) Twin-axis time series.



(b) Phase portrait.

Figure 5.3: Example simulation with no escape.

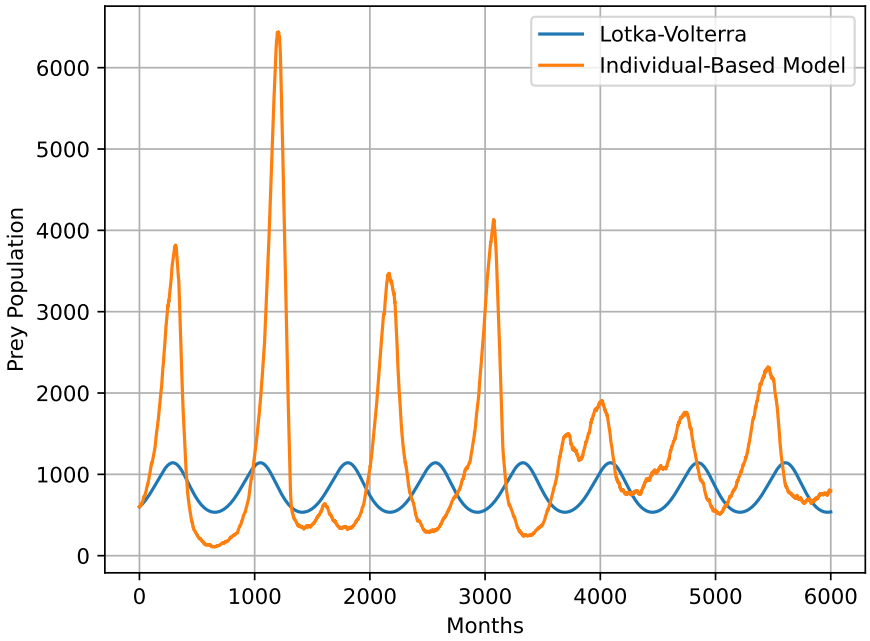
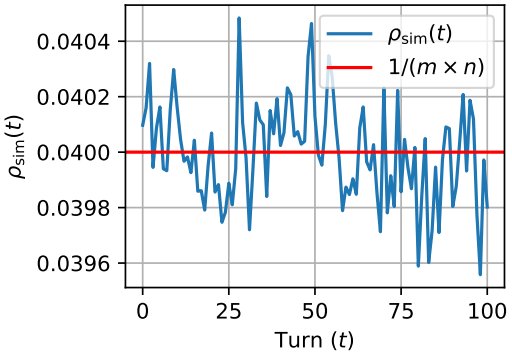
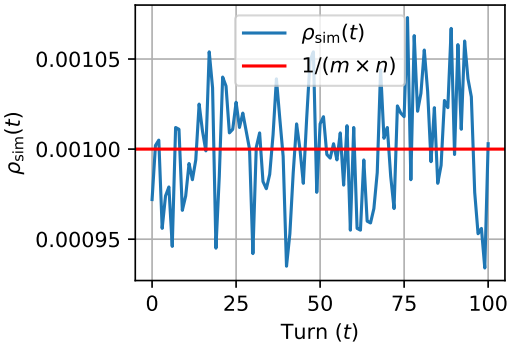


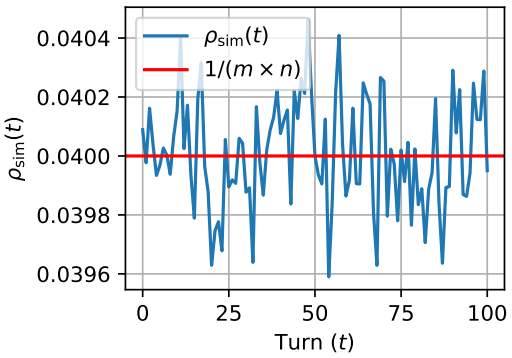
Figure 5.4: Prey population dynamics of the Lotka-Volterra solution and an IBM simulation.



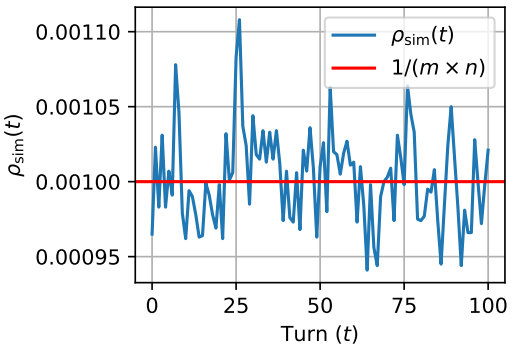
(a) A 5×5 grid with no escape.



(b) A 10×100 grid with no escape.

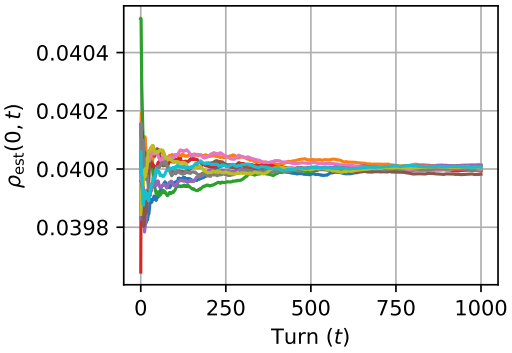


(c) A 5×5 grid with random edge point escape.

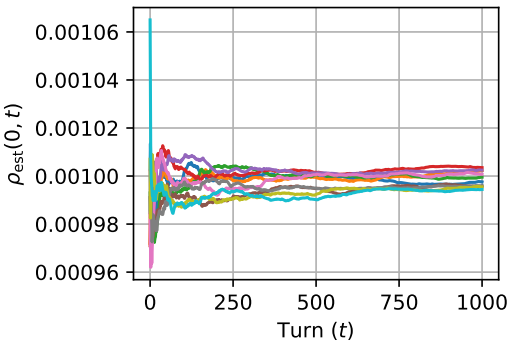


(d) A 10×100 grid with random edge point escape.

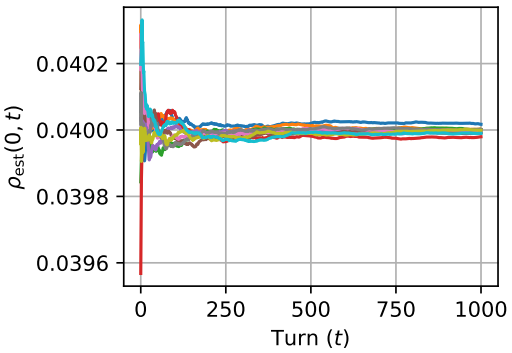
Figure 5.5: Change in $\rho_{\text{sim}}(t)$ over time.



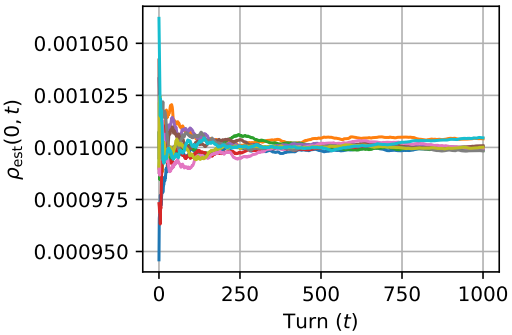
(a) A 5×5 grid with no escape.



(b) A 10×100 grid with no escape.

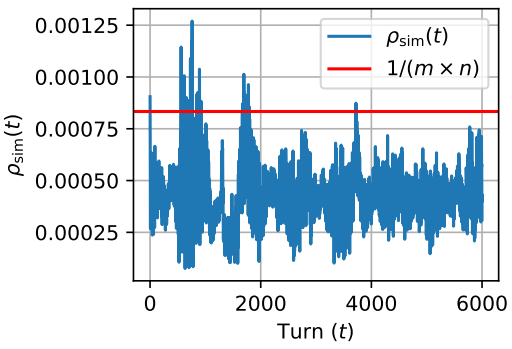


(c) A 5×5 grid with random edge point escape.

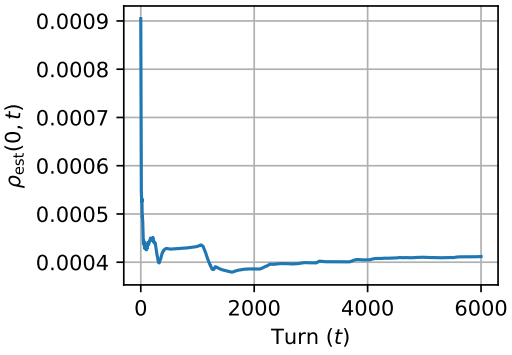


(d) A 10×100 grid with random edge point escape.

Figure 5.6: Change in $\rho_{\text{est}}(0,t)$ over time.

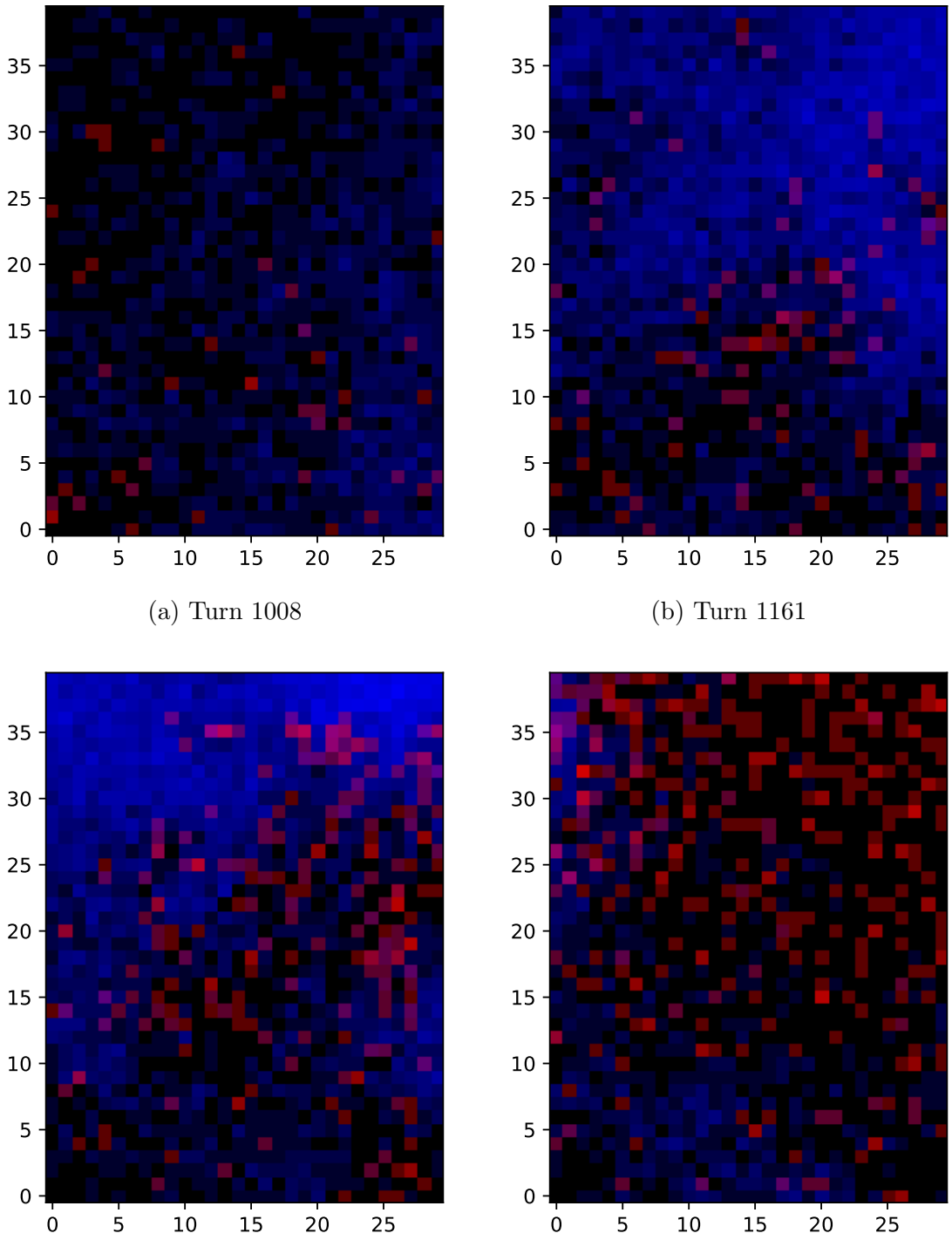


(a) Change in $\rho_{\text{sim}}(t)$ over time.



(b) Change in $\rho_{\text{est}}(0,t)$ over time.

Figure 5.7: Change in ρ measurements over time in our simulation example.



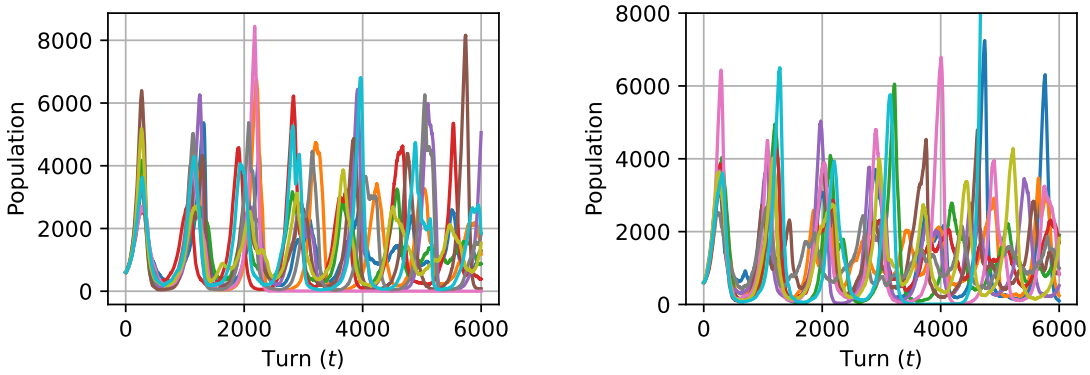
(a) Turn 1008

(b) Turn 1161

(c) Turn 1231

(d) Turn 1310

Figure 5.8: Point maps showing pattern change over time.



(a) Different initial grid layout.

(b) Same initial grid layout.

Figure 5.9: Time series of the prey populations for 10 runs of the example simulation.

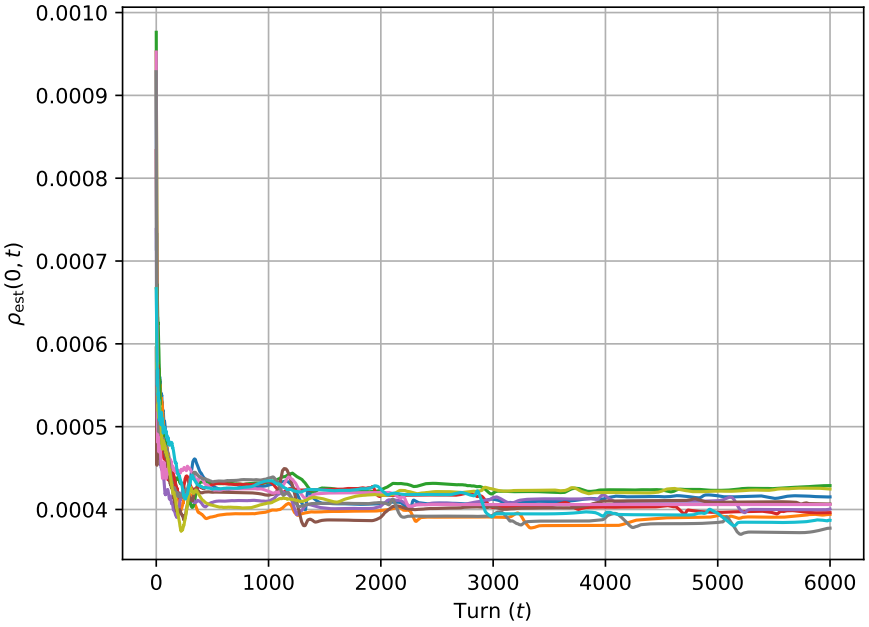


Figure 5.10: $\rho_{\text{est}}(0,t)$ for 10 runs of the example simulation with no escape.

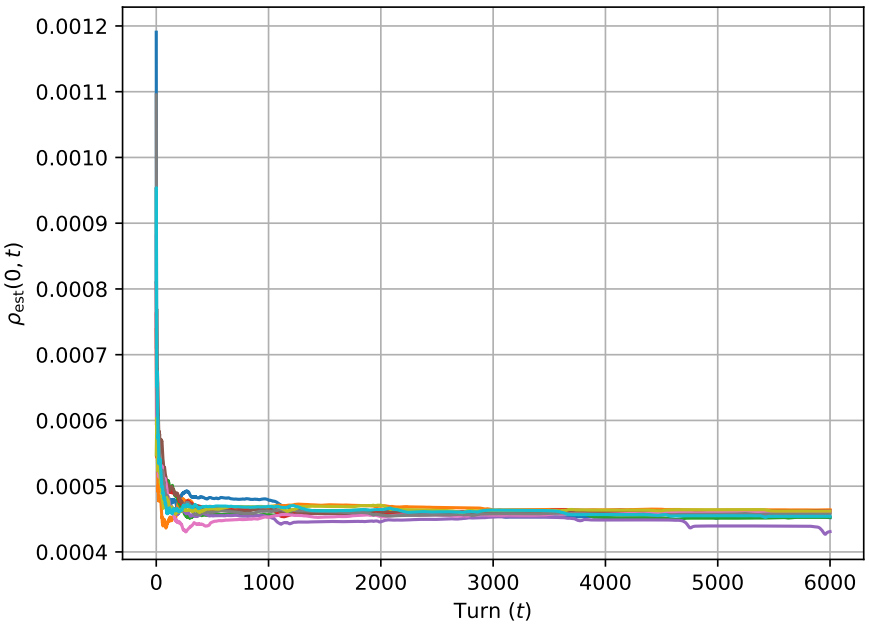


Figure 5.11: $\rho_{\text{est}}(0,t)$ for 10 runs of the example simulation with random edge point escape.

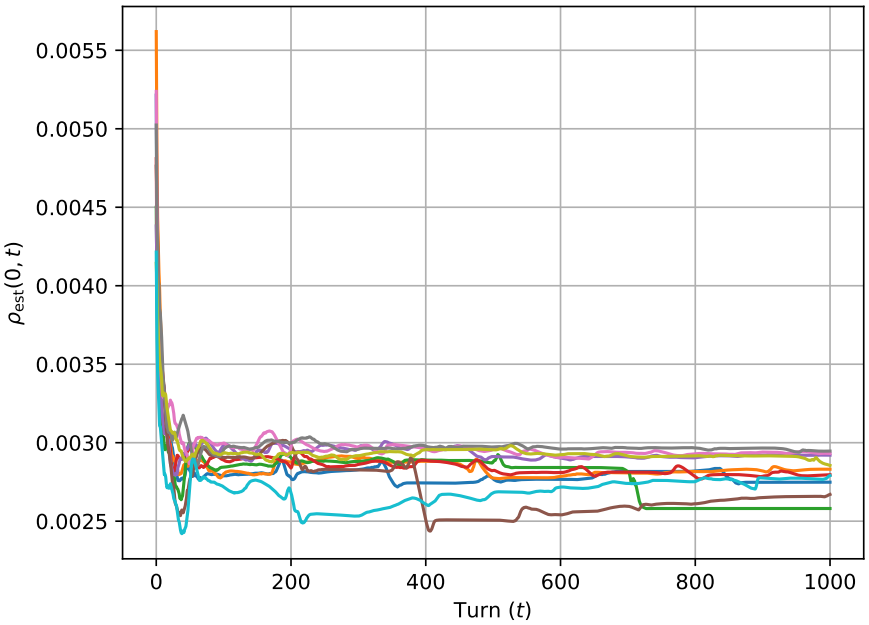


Figure 5.12: $\rho_{\text{est}}(0,t)$ for 10 runs of the example simulation with a turn corresponding to six months.

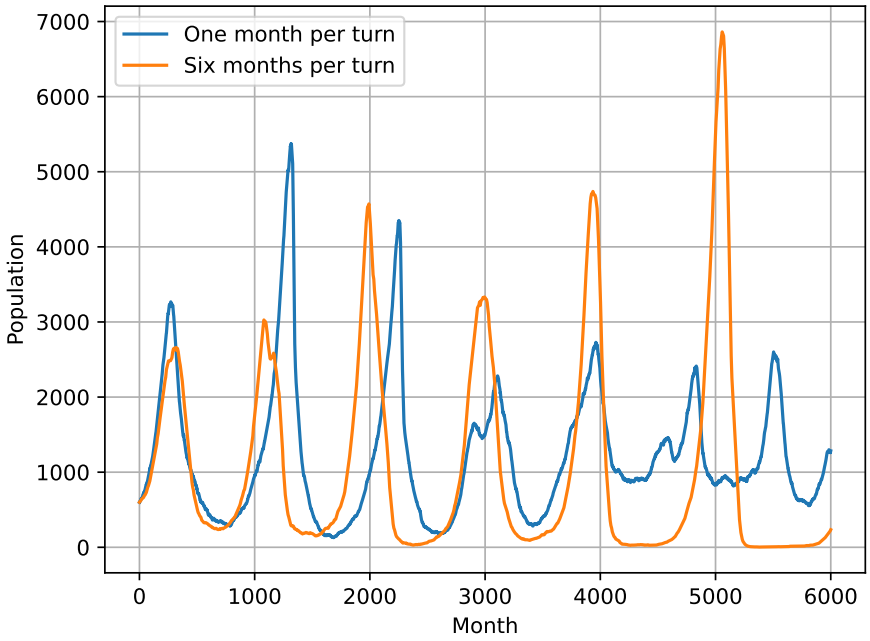


Figure 5.13: A time series of two simulations' prey populations with one being run with a turn corresponding to one month and the other six months.

Chapter 6

Discussion

In this report we have taken small steps to eventually convert the Lotka-Volterra equations into an individual-based model. The point of this is to challenge what the equations represent, by producing something more tangible out of them: a simulation that you can watch occur. This report does not state for certain that the IBM model is a better model of reality, as this cannot be said without real-world experimentation. Yet this report lays out the utility that a 'simulation-twin' can be to evaluating an analytical model. It gives you a different tool for analysis.

6.1 Spiralling to Extinction Problem

A problem with our simulation is the tendency for prey and predator populations to spiral away from equilibrium and eventually to extinction. An example of this is the third experiment of the six-month-per-turn simulation. The phase portrait plotted in figure 6.2 shows how the population sizes spiral out, and the time series 6.1 displays this as the populations' wave heights increasing. This can occur because there is no conserved quantity like there is in the Lotka-Volterra equations, so the wave heights are able to change which they are not in Lotka-Volterra solutions.

If we study the time series, we see that a cycle is occurring: low number of prey causes a large number of predator deaths; this cause low numbers of predators meaning prey populations sky rocket; then predator populations benefit greatly by the large prey populations, causing very low prey numbers. This cycle can cause wave heights to increase until there are too many predators, and the prey get killed off.

This is a regular occurrence in these simulations, yet extinction is not as common in nature as the model claims. This disconnect can be blamed on the simulation failing to integrate limiting factors. For example, in nature, rapid growth of a

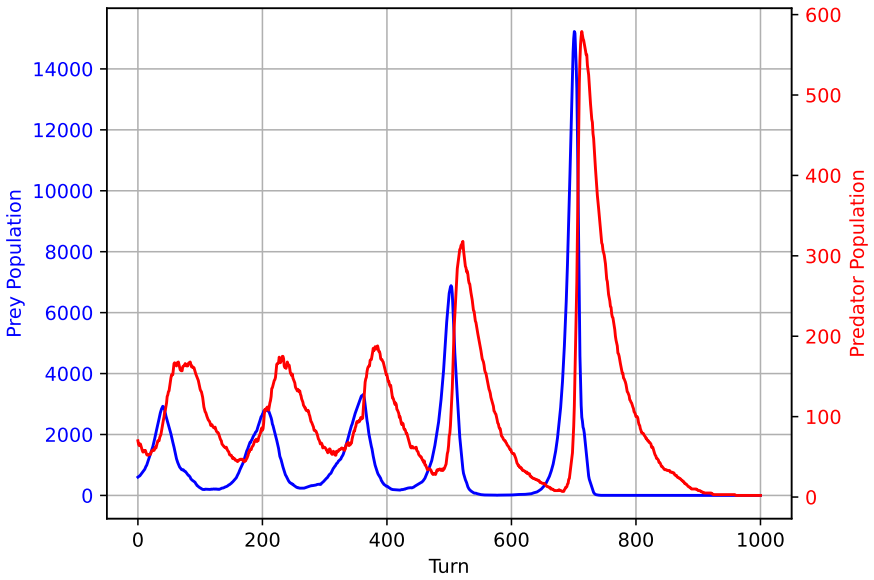


Figure 6.1: Twin-axis time series of the prey and predator populations.

species in an area is curbed by the available sustenance available for that species. The number of animals that an environment can sustain is called its carrying capacity. Our model does not integrate carrying capacity for prey (though it does for predators as prey is its food source). Therefore, prey populations are able to grow exponentially in times when there are not many predators. In reality, the population would round out as it reaches the carrying capacity of the area. We could improve the model by adding a food source for prey. We could say that each grid point grows food at a specific rate up to a certain point (its own carrying capacity), and prey have to eat a certain amount of food to reproduce, or has a percentage chance of reproduction when it eats some food to keep it simple in the same way predators reproduce. This would stop over-production of prey in certain areas as they would use up their food source quickly and then their reproduction would slow.

Another limiting factor that the model does not integrate is the cap on reproduction rates for a predator. If a predator is near many prey, it may be eating and reproducing at a rapid rate, because there is no limit to how many prey a predator can eat. In reality, accumulating biomass and reproducing is a more measured occurrence; there is a maximum reproduction rate for an individual, no matter how rich the food source is.

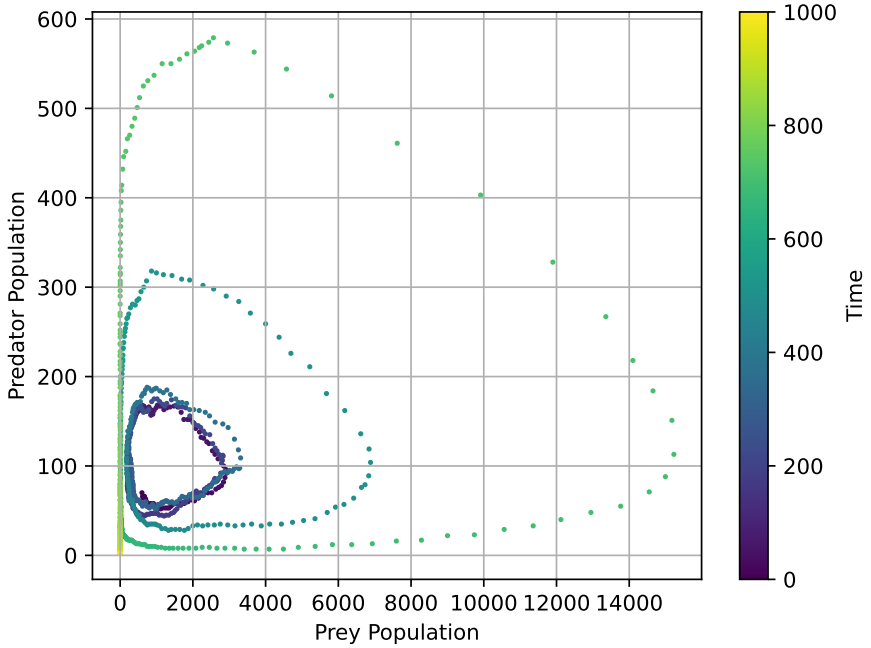


Figure 6.2: Phase portrait of the prey and predator populations

6.2 Utility of the Model

In this report we have attempted to build a simulation which reflects the world that the Lotka-Volterra equations model. The Lotka-Volterra model assumes a simple situation, which helps when defining and analysing the model. However, the assumptions that it makes to achieve this simplicity are arguably unrealistic.

Our simulation fails to agree with the assumptions made by LV. In particular, our simulation disagrees with the assumption that animals in a habitat are well-mixed. Our ρ_{sim} values in different simulations were consistently lower than the value if the habitats were well-mixed, ρ. Weighted averages of ρ_{sim}, ρ_{est}, were consistently around half of ρ, telling us that the chance of a prey and predator meeting was about half the what the Lotka-Volterra model would expect it to be.

As we saw in our comparison of the two models' results, this makes a sizeable difference to how the populations of the species change. Our simulation gives a much larger wave height for the population oscillations than LV does.

With a lack of accompanying real-world data, it is difficult to say with confidence which model is more realistic. Data available showing predator-prey population changes do not give a reliable picture of the dynamics. This is because of the difficulty of scientifically counting the population sizes over long time periods. An experiment would have to span centuries to give us enough data to work with,

depending on the expected life spans of the predators and prey. Further, a situation where a prey species only has one predator, and vice versa is uncommon in nature.

This is the reason why building simulations has utility. We can in milliseconds simulate an animal's life which may in reality go on for years, giving us access to data much more quickly. This data can then be used to evaluate simpler models, like Lotka-Volterra. The issue comes when telling whether the simulation itself is accurate. This is a hard problem to solve, but iterating on a simulation and adding our knowledge of the context to it gets us closer to an accurate simulation.

If the simulation in the report is iterated on further by adding contextual knowledge such as adding a carrying capacity for prey and a limit to reproduction rates, we would expect the simulation to be more accurate. We would evaluate the new simulation using our understanding of the context to tell how accurate it is. We may then decide to further increase the accuracy by giving animals movement choice. For example, a predator may choose to move to an adjacent grid where there is a prey.

Once this simulation has been repeatedly iterated upon so that it is believed to be an accurate model of reality, it can be used to evaluate simpler models like Lotka-Volterra in a way that you would normally use real-world data to evaluate a model. In this evaluation, one would decide whether the simpler model captures the important characteristics of the simulation, in the same way that a mathematical modeller attempts to capture the relevant characteristics of what they are modelling.

We did not get far enough to make a judgement on the validity of the Lotka-Volterra equations, but we did show how one can pursue such a question.

Appendix A

Mathematical Definitions

A.1 The Derivative of a Function

As written by citation [3, Section 3.1], let $f(x)$ be a function defined in an open interval containing a. The derivative of the function $f(x)$ at a, denoted by $f'(a)$, is defined by

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}. \tag{A.1}$$

Appendix B

Code

Listing B.1: Discrete Time Model

```
def discrete_time (
    p: dict
) -> tuple [list [float], list [float], list [int] ]:

    """Time is modelled as turn-based while prey x and
    predator y populations are modelled as
    continuous values."""

    x = float (p ['x0'] )
    y = float (p ['y0'] )

    xs = [x]
    ys = [y]

    ts = list (range (0, p ['num_turns'] + 1) )

    prey_extinct = False

    for i in ts [1:]:

        # - Prey Death -

        amount_x_die = min (x, p ['beta'] * x * y)

        if amount_x_die == x and not prey_extinct:
            prey_extinct = True
```

```
            print (f'Prey extinct at turn {i}')

        Delta_x_death = - amount_x_die
        x += Delta_x_death

        # - Predator Reproduction -

        y += (1 / p ['sigma'] ) * (- Delta_x_death)

        # - Prey Reproduction -

        x += p ['alpha'] * x

        # - Predator Death -

        Delta_y_death = -
        y -= p ['gamma'] * y

        xs. append (x)
        ys. append (y)

    return xs, ys, ts
```

Listing B.2: Stochastic Rounding

```
def round_stochastically (
    f: float,
    rng: Generator,
) -> int:

    """Stochastically rounds floats and
    returns as an integer."""

    integer_part = int (f)
    fractional_part = f - integer_part

    draw = rng. random ()
    prob_round = 1 if draw < fractional_part else 0

    number = integer_part + prob_round

    return number
```

Listing B.3: Discrete Time-and-Animals Model

```
def discrete_everything (
        p: dict, rng: Generator
    ) -> tuple [list [int], list [int], list [int] ]:

    """Discrete animals and time."""

    x = p ['x0']
    y = p ['y0']

    xs = [x]
    ys = [y]

    prey_extinct = False

    num_turns = p ['num_turns']

    for i in range (1, num_turns + 1):

        # - Prey Death -

        num_x_die = min (x, round_stochastically (
            f = p ['beta'] * x * y,
            rng = rng,
            ) )

        if num_x_die == x and not prey_extinct:

            prey_extinct = True
            print (f'Prey␣extinct␣at␣turn␣{i}')

        Delta_x_death = - num_x_die
        x += Delta_x_death

        # - Predator Reproduction -

        Delta_y_reproduction = round_stochastically (
            f = (1 / p ['sigma'] ) * (- Delta_x_death),
            rng = rng
        )
```

```
        y += Delta_y_reproduction

        # - Prey Reproduction -

        Delta_x_reproduction = round_stochastically (
            f = p ['alpha'] * x,
            rng = rng
        )
        x += Delta_x_reproduction

        # - Predator Death -

        num_y_die = min (y, round_stochastically (
            f = p ['gamma'] * y,
            rng = rng,
        ) )

        if num_y_die == y:

            print (f'Predators␣extinct␣at␣turn␣{i}')
            num_turns = i

            xs. append (x)
            ys. append (0)

            break # Break on predator extinction
            # else the prey will exponentially increase

        Delta_y_death = - num_y_die
        y += Delta_y_death

        xs. append (x)
        ys. append (y)

    ts = list (range (0, num_turns + 1) )

    return xs, ys, ts
```

Listing B.4: Simple Individual-Based Model

```
def ibm (
        p: dict,
```

```
        rng: Generator
) -> tuple [list [int], list [int], list [int] ]:

    "Each␣animal␣treated␣individually."

    x = p ['x0']
    y = p ['y0']

    xs = [x]
    ys = [y]

    num_turns = p ['num_turns']

    prey_extinct = False

    for i in range (1, num_turns + 1):

        # - Prey Death -

        num_x_die = min (
            x, rng. binomial (
                n = x * y, p = p ['beta']
            ) )

        if num_x_die == x and not prey_extinct:
            prey_extinct = True
            print (f'Prey␣extinct␣at␣turn␣{i}')

        Delta_x_death = - num_x_die
        x += Delta_x_death

        # - Predator Reproduction -

        Delta_y_reproduction = rng. binomial (
            n = (- Delta_x_death),
            p = (1 / p ['sigma'] )
        )
        y += Delta_y_reproduction

        # - Prey Reproduction -
```

```
        Delta_x_reproduction = rng. binomial (
            n = x,
            p = p ['alpha']
        )
        x += Delta_x_reproduction

        num_y_die = rng. binomial (
            n = y,
            p = p ['gamma']
        )

        # - Predator Death -

        if num_y_die == y:

            print (f'Predators␣extinct␣at␣turn␣{i}')
            num_turns = i

            xs. append (x)
            ys .append (0)

            break

        Delta_y_death = - num_y_die
        y += Delta_y_death

        xs. append (x)
        ys. append (y)

    ts = list (range (0, num_turns + 1) )

    return xs, ys, ts
```

Bibliography

[1] Jeffrey R. Chasnov. *Mathematical Biology*. Hong Kong University of Science and Technology, 2025.

[2] Matteo Croci et al. "Stochastic rounding: implementation, error analysis and applications". In: *Royal Society Open Science* 9.3 (Mar. 2022), p. 211631. ISSN: 2054-5703. DOI: 10.1098/rsos.211631.

[3] Gilbert Strang and Edwin Herman. *Calculus Volume 1*. OpenStax, 2016.

[4] William F. Trench. *Euler's Method*. Trinity University. 2020. URL: https://math.libretexts.org/@go/page/44234 (visited on 05/2026).

[5] Vito Volterra. "Fluctuations in the Abundance of a Species considered Mathematically". In: *Nature* 118 (1926), pp. 558–560.